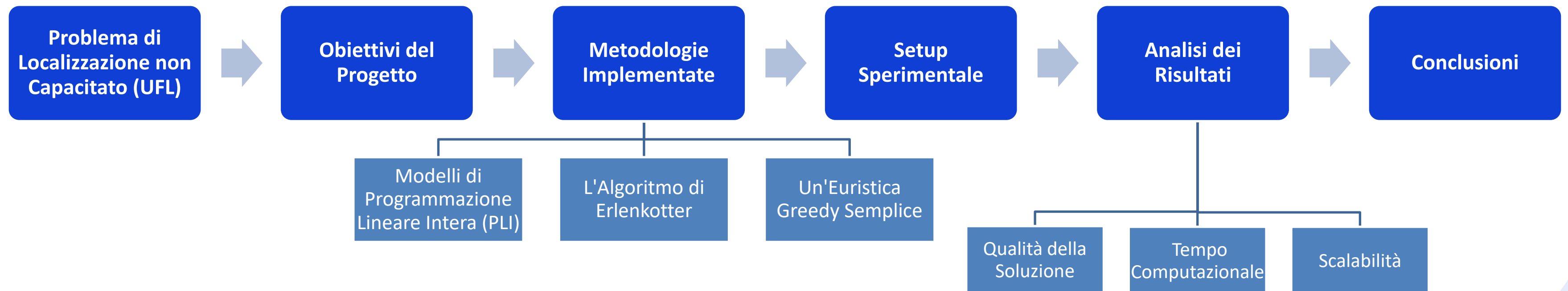


# **Risoluzione del Problema Uncapacitated Facility Location (UFL)**

**Analisi Comparativa di Modelli Esatti ed Euristiche**

Valentina Jin  
0365400  
A.A. 2024/2025

# AGENDA



# Problema UFL

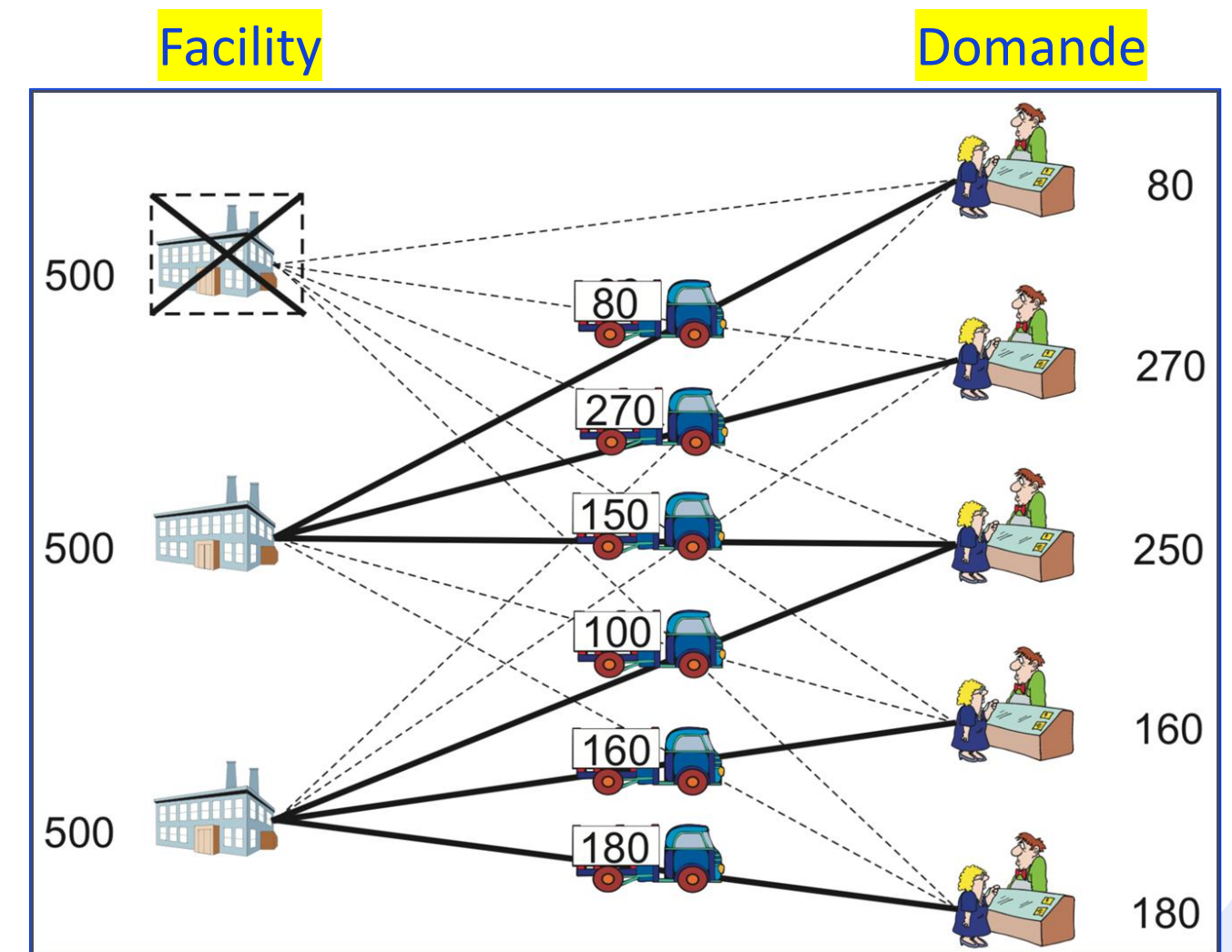
**Domanda chiave:** "Dato un insieme di clienti e un insieme di possibili sedi per dei magazzini, dove conviene aprire i magazzini per servire tutti i clienti al costo più basso possibile? "

- Costi Fissi (di Apertura): Un costo  $f_u$  da pagare se si decide di aprire il magazzino  $u$ .
- Costi Variabili (di Assegnazione): Un costo  $c_{uv}$  da pagare per servire il cliente  $v$  dal magazzino  $u$ .

**Obiettivo:** Minimizzare la somma dei costi fissi totali e dei costi di assegnazioni totali.

**Vincoli:**

- Ogni cliente deve essere servito da esattamente un magazzino.
- Un cliente può essere servito solo da un magazzino aperto.



# Problema UFL

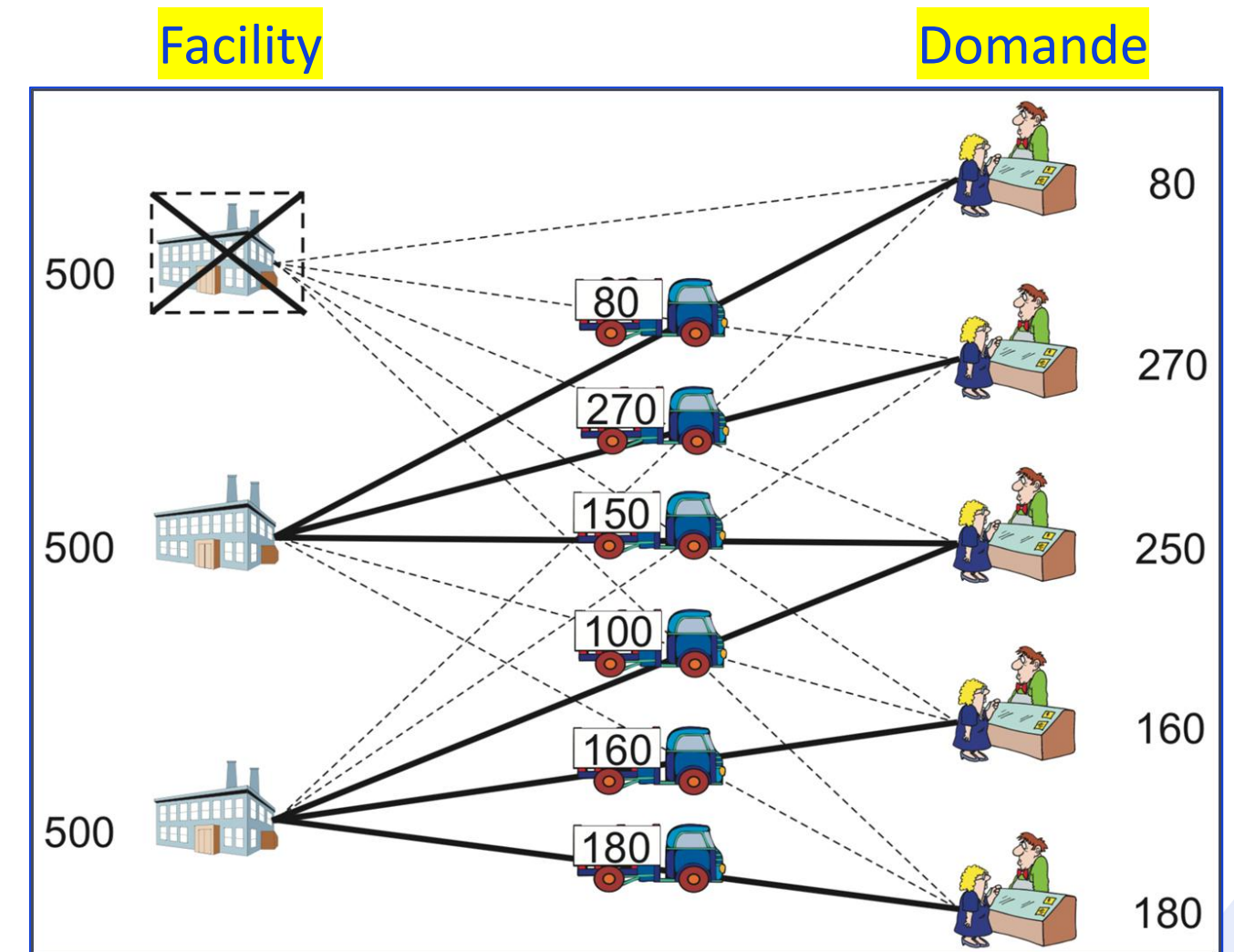
## INPUT:

- Facility
- Client
- Costi fissi
- Costi di assegnazione



## OUTPUT:

- Costo totale



# Obiettivi del Progetto

01

Implementare e analizzare diversi approcci risolutivi per il problema UFL:

- Metodi esatti basati sulla Programmazione Lineare Intera (PLI).
- Un'euristica classica (Algoritmo di Erlenkotter).
- Un'euristica semplice di costruzione (Greedy).

02

Confrontare le **performance** dei diversi metodi secondo metriche standard.

03

Valutare il compromesso (**trade-off**) tra qualità della soluzione (vicinanza all'ottimo) e tempo computazionale.

04

Studiare la **scalabilità** degli algoritmi, osservando come si comportano all'aumentare della dimensione delle istanze.



# Metodi Esatti (PLI)

Forniscono la  
soluzione  
ottima

Funzione obiettivo:

$$z = \min \sum_{u \in S} f_u \cdot x_u + \sum_{u \in S} \sum_{v \in D} c_{uv} \cdot y_{uv} = \text{cost}(x, y)$$

Vincoli:

$$\sum_{u \in S} y_{uv} = 1 \quad \forall v \in D$$

$$y_{uv} \leq x_u \quad \forall u, v \in S \times D$$

**Formulazione forte**

$$\sum_{v \in D} y_{uv} \leq q \cdot x_u \quad \forall u \in S$$

**Formulazione debole**

Variabili:

$$x_u \in \{0,1\} \quad y_{uv} \in \{0,1\} \quad \forall u, v \in S \times D$$



Fornisce il  
lower bound

# Algoritmo di Erlenkötter (DUALOC)

DUALOC è un **algoritmo dual ascent**: si lavora sul *problema duale dell'UFL*, aumentando progressivamente le variabili duali fino a saturare i vincoli.

**Funzione obiettivo (duale):**

$$w = \max \sum_{v \in D} \mu_v$$

Massimizzare la somma dei prezzi che i clienti sono disposti a pagare per essere serviti.

**Vincoli (duali):**

$$\sum_{v \in D} \lambda_{uv} \leq f_u \quad \forall u \in S$$

Le facility possono essere “finanziate” solo fino al loro costo di apertura.

$$\mu_v - \lambda_{uv} \leq c_{uv} \quad \forall u, v \in S \times D$$

Un cliente non può “pagare” più di quanto costa servirlo.

**Variabili (duali):**

$$\mu_v \geq 0, \lambda_{uv} \geq 0 \quad \forall u, v \in S \times D$$

I contributi non possono essere negativi.

# Algoritmo di Erlenkötter (DUALOC)

Nell'implementazione di questo algoritmo, invece di determinare direttamente il valore ottimale di ciascuna variabile duale  $\mu$  in un unico passo (come accadrebbe in un approccio coordinate ascent), l'algoritmo di **Dual Ascent** raggiunge lo stesso obiettivo con una **strategia incrementale**: per ogni variabile duale viene calcolato un incremento (Delta), che viene poi aggiunto al valore corrente, aggiornando progressivamente la soluzione duale.

## Fasi principali:

### 1. Fase inizializzazione:

- A ogni cliente  $j$  viene associato un "prezzo"  $v_j$ , inizialmente pari al costo minimo di trasporto da una qualsiasi facility.
- Per ogni potenziale facility  $i$ , si calcola il *profitto duale*  $s_i$ , che rappresenta il suo costo fisso non ancora coperto dai "pagamenti" dei clienti.
- Le facility per cui  $s_i \geq 0$  vengono considerate *attive* o *candidate* all'apertura.

Corrisponde a  $\mu_v$

Corrisponde a  $f_i - \sum_v \lambda_{iv}$

```
# Step 1: Initialization
# Inizializziamo v_j al costo della facility più economica per il cliente j (c_j^1)
v = sorted_costs[0, :].copy() # Usiamo .copy() per evitare modifiche inattese

# Calcoliamo il surplus (slack) iniziale s_i per ogni facility
savings = np.maximum(x1: 0, v - transport_costs).sum(axis=1)
surplus = fixed_costs - savings

# k_j tiene traccia dell'indice del "prossimo miglior costo" per il cliente j.
# In 0-based indexing, k=1 significa il secondo costo più basso (sorted_costs[1, j]).
k_j = np.ones(num_customers, dtype=int)

# Gestione delle parità (ties): se v_j è già uguale al prossimo miglior costo, incrementiamo k_j
for j in range(num_customers):
    while k_j[j] < num_facilities and np.isclose(v[j], sorted_costs[k_j[j], j]):
        k_j[j] += 1
```



# Algoritmo di Erlenkotter (DUALOC)

```
# Step 2: Loop principale
while True:
    delta_flag = False

    # Loop sui clienti j = 0 a n-1
    for j in range(num_customers):

        # Step 3: Se non possiamo più aumentare v_j, saltiamo il cliente
        if k_j[j] >= num_facilities:
            continue

        # Step 4: Calcola il primo candidato per l'aumento Delta_j
        # Troviamo le facility 'i' che contribuiscono a v_j (dove v_j >= c_ij)
        contributing_mask = (v[j] - transport_costs[:, j]) >= -1e-9 # Tolleranza per float

        if not np.any(contributing_mask):
            # Caso raro, se nessuna facility contribuisce, non possiamo aumentare basandoci sul surplus
            delta_j = np.inf
        else:
            # Delta_j è il surplus minimo tra le facility che contribuiscono
            delta_j = np.min(surplus[contributing_mask])

        # Step 5: Limita Delta_j in base al prossimo miglior costo
        gap_to_next_cost = sorted_costs[k_j[j], j] - v[j]
```

## 2. Procedura di Ascesa Iterativa:

- L'algoritmo itera ripetutamente su tutti i clienti.
- Per ogni cliente  $j$ , si calcola l'aumento massimo possibile ( $\Delta_j$ ) per il suo prezzo  $v_j$ , rispettando due condizioni:
  1. Non deve rendere negativo il surplus  $s_i$  di nessuna facility.
  2. Non deve superare il costo della "prossima facility più economica" per quel cliente.
- Il prezzo  $v_j$  viene aumentato di  $\Delta_j$  e i surplus delle facility coinvolte vengono aggiornati.

# Algoritmo di Erlenkötter (DUALOC)

## 3. Terminazione e Risultato:

- La procedura termina quando nessun prezzo  $v_j$  può più essere aumentato in un'intera iterazione.
- Il Lower Bound è dato dalla somma finale di tutti i prezzi duali  $v_j$ .

```
if delta_j > gap_to_next_cost:
    delta_j = gap_to_next_cost
    # Se abbiamo modificato delta_j, incrementiamo k_j per il prossimo giro
    # e segnaliamo che una modifica importante è avvenuta in questo passo
    if delta_j > 1e-9:
        delta_flag = True
        k_j[j] += 1

# Step 6: Applica l'aumento Delta_j
if delta_j > 1e-9:
    # Diminuisci il surplus per le facility che contribuiscono
    surplus[contributing_mask] -= delta_j
    # Aumenta la variabile duale v_j
    v[j] += delta_j
    # Qualsiasi aumento di v_j significa che il processo non è ancora stabile
    delta_flag = True

# Step 8: Se nessuna variabile v_j è stata modificata in un intero ciclo, termina.
# Altrimenti, ricomincia dal passo 2 (il nostro `while True` loop).
if not delta_flag:
    break

# --- FINE DUAL ASCENT ---

# --- CALCOLO DEL LOWER BOUND DUALE ---
# Il valore dell'obiettivo duale è la somma dei v_j
dual_lower_bound = np.sum(v)
```

# Euristica Greedy



## Come funziona:

### 1. Inizializzazione:

- Si parte da una soluzione vuota, senza nessuna facility aperta.
- Il costo iniziale è considerato infinito.

### 2. Procedura Iterativa:

- Ad ogni passo, l'algoritmo valuta tutte le facility non ancora aperte.
- Per ciascuna di esse, calcola il **risparmio di costo totale** che si otterrebbe aprendola. Questo risparmio è dato da:

$(\text{Riduzione dei costi di trasporto}) - (\text{Costo fisso della nuova facility})$

- Decisione "Greedy": Se il massimo risparmio possibile è positivo, l'algoritmo apre la facility che offre il risparmio più alto.

### 3. Terminazione e Risultato:

- La procedura termina quando nessuna facility chiusa può offrire un risparmio positivo.
- L'insieme delle facility aperte e il costo totale associato costituiscono la soluzione finale (l'upper bound).

# ARCHITETTURA

## Tool

- Linguaggio di programmazione: Python
- IDE: PyCharm
- Solver AMPL: Gurobi

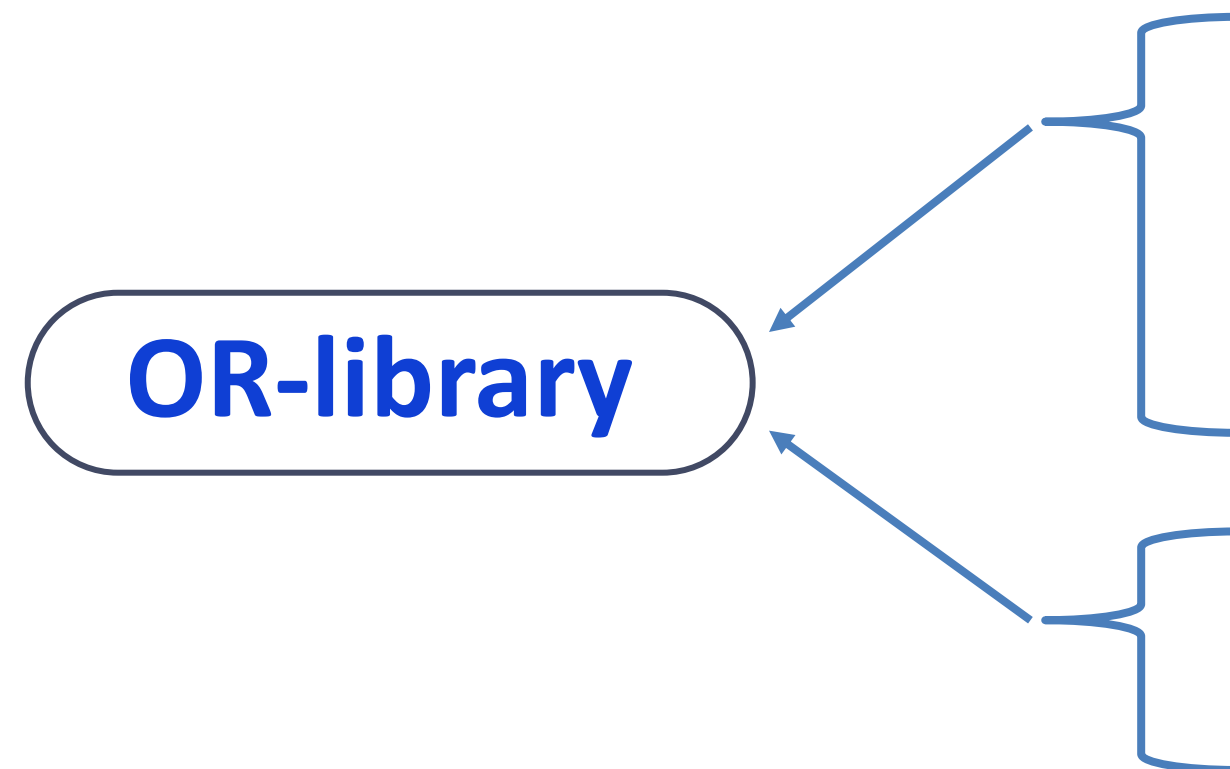
## Struttura

Come ho strutturato il codice:

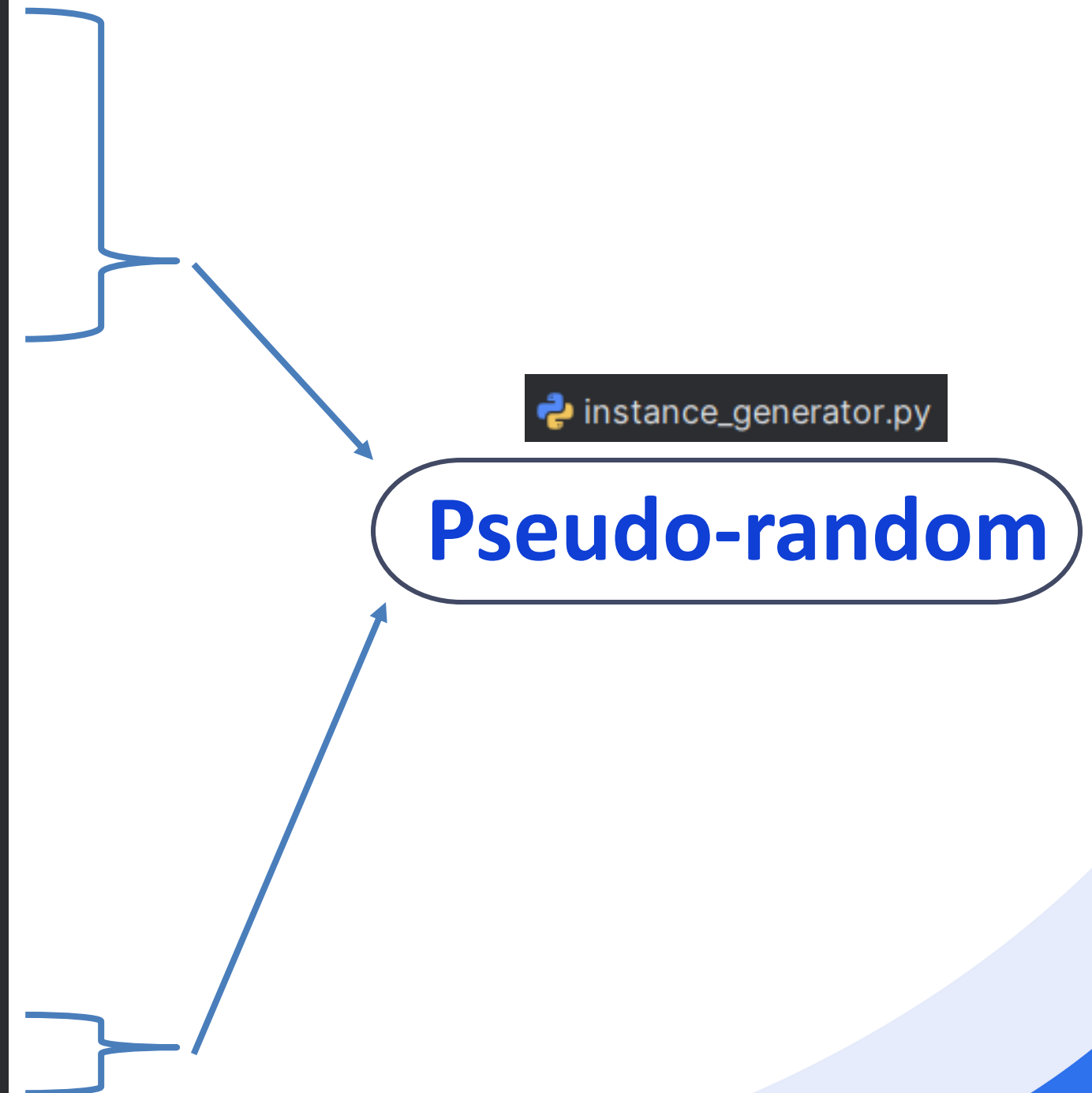
- *data*: istanze di test
- *reports*: analisi dettagliate per ciascuna istanza
- *src*: algoritmi Greedy, Erlenkotter, modelli PLI
- *utilis*: programmi per generare i plot

# ISTANZE e DATI

| Size  | Facility   | Client     | Istanze totali |
|-------|------------|------------|----------------|
| Big   | $\leq 100$ | $\leq 250$ | $\leq 22500$   |
| Medio | $\leq 50$  | $\leq 50$  | $\leq 2500$    |
| Small | $\leq 25$  | $\leq 50$  | $\leq 1250$    |



```
data
├── big
│   ├── gen_f75_c150_s123.txt
│   ├── gen_f75_c200_s123.txt
│   ├── gen_f80_c150_s123.txt
│   ├── gen_f90_c200_s123.txt
│   ├── gen_f90_c250_s123.txt
│   ├── gen_f100_c150_s123.txt
│   └── gen_f100_c200_s123.txt
├── medio
│   ├── cap122.txt
│   ├── cap123.txt
│   ├── cap124.txt
│   ├── cap131.txt
│   ├── cap132.txt
│   ├── cap133.txt
│   └── cap134.txt
└── small
    ├── cap42.txt
    ├── cap62.txt
    ├── cap71.txt
    ├── cap72.txt
    ├── cap84.txt
    ├── gen_f10_c30_s123.txt
    └── gen_f25_c25_s123.txt
```





# CRITERIO DI ANALISI

**Costo della funzione  
obiettivo finale**

**GAP rispetto al costo  
ottimo**

**Tempo di  
computazione**

**Numero di facility  
aperti / Numero  
totale di facility**

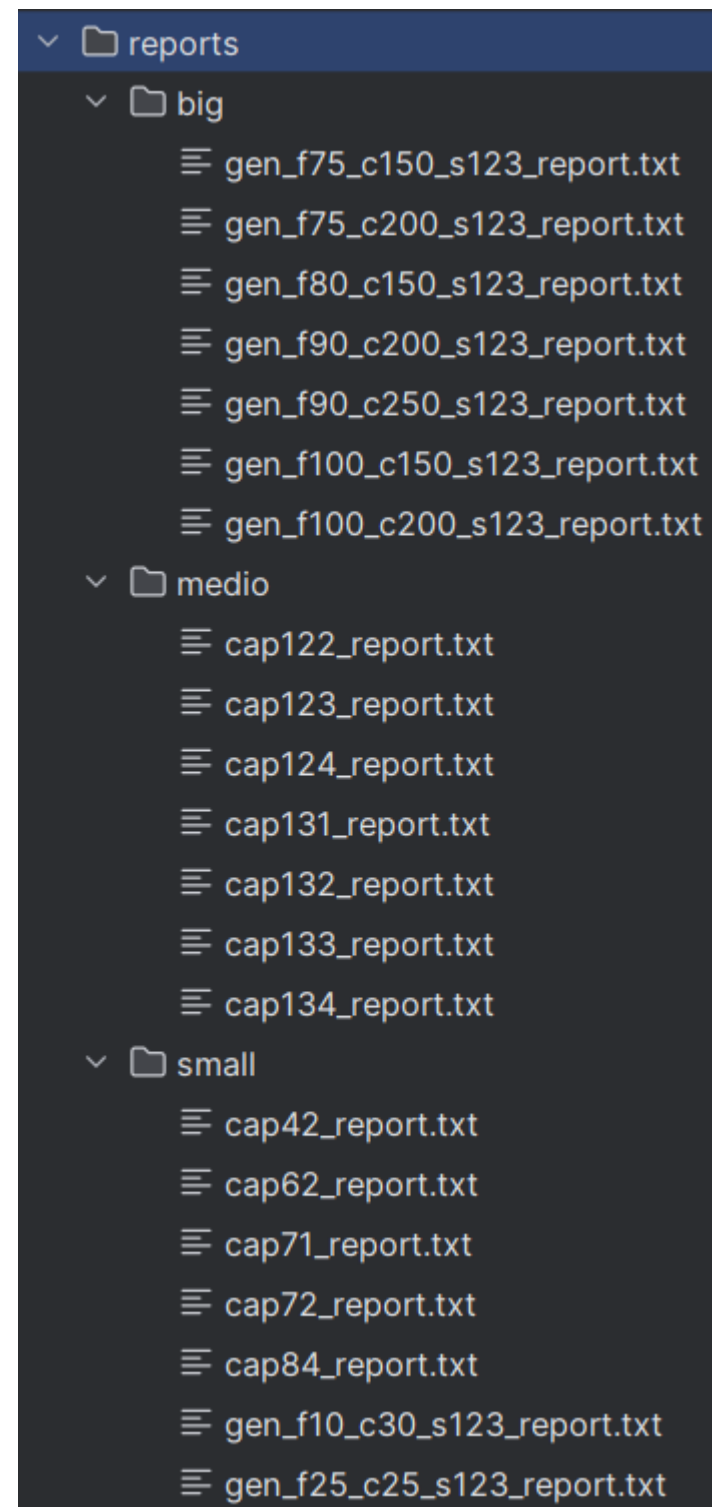
# Analisi di Gap

Consideriamo:

1. **Gap di Ottimalità** (per Upper Bounds):  $(z_{\text{upper}} - z^*) / z^*$ 
  - Misura di quanto una soluzione ammissibile (ma sub-ottima) è peggiore dell'ottimo.
  - È un valore  $\geq 0\%$ . Si applica all'euristica greedy.
2. **Gap di Dualità** (per Lower Bounds):  $(z^* - z_{\text{lower}}) / z^*$ 
  - Misura quanto un limite inferiore è lontano dal valore ottimo. Ci dice quanto è "stretta" la stima fornita dal rilassamento o dall'algoritmo duale.
  - È anch'esso un valore  $\geq 0\%$ . Si applica a strong\_rl, weak\_rl e erlenkotter.

# Analisi di Performance per le singole istanze small

# Analisi di Performance per le singole istanze dettagliate



Esiste un report dettagliato per  
ciascun istanza testata, ad  
esempio per l'istanza cap72.txt:

```
=====
ANALYSIS REPORT FOR INSTANCE: cap72.txt
Generated on: 2025-11-03 11:08:14
=====

Instance Details:
- Facilities: 16
- Customers: 50

--- Algorithm: strong ---
- Objective Cost:          977799.4000000001
- Computational Time (s):  0.0498
- Opened Facilities Count: 9
- Opened Facilities (IDs): [0, 1, 2, 3, 5, 6, 7, 10, 12]

--- Algorithm: weak ---
- Objective Cost:          977799.4000000001
- Computational Time (s):  0.0549
- Opened Facilities Count: 9
- Opened Facilities (IDs): [0, 1, 2, 3, 5, 6, 7, 10, 12]

--- Algorithm: strong_rl ---
- Objective Cost:          977799.4000000001
- Computational Time (s):  0.0534
- Optimality Gap:          0.0%

--- Algorithm: weak_rl ---
- Objective Cost:          849169.0375000003
- Computational Time (s):  0.0535
- Optimality Gap:          13.16%

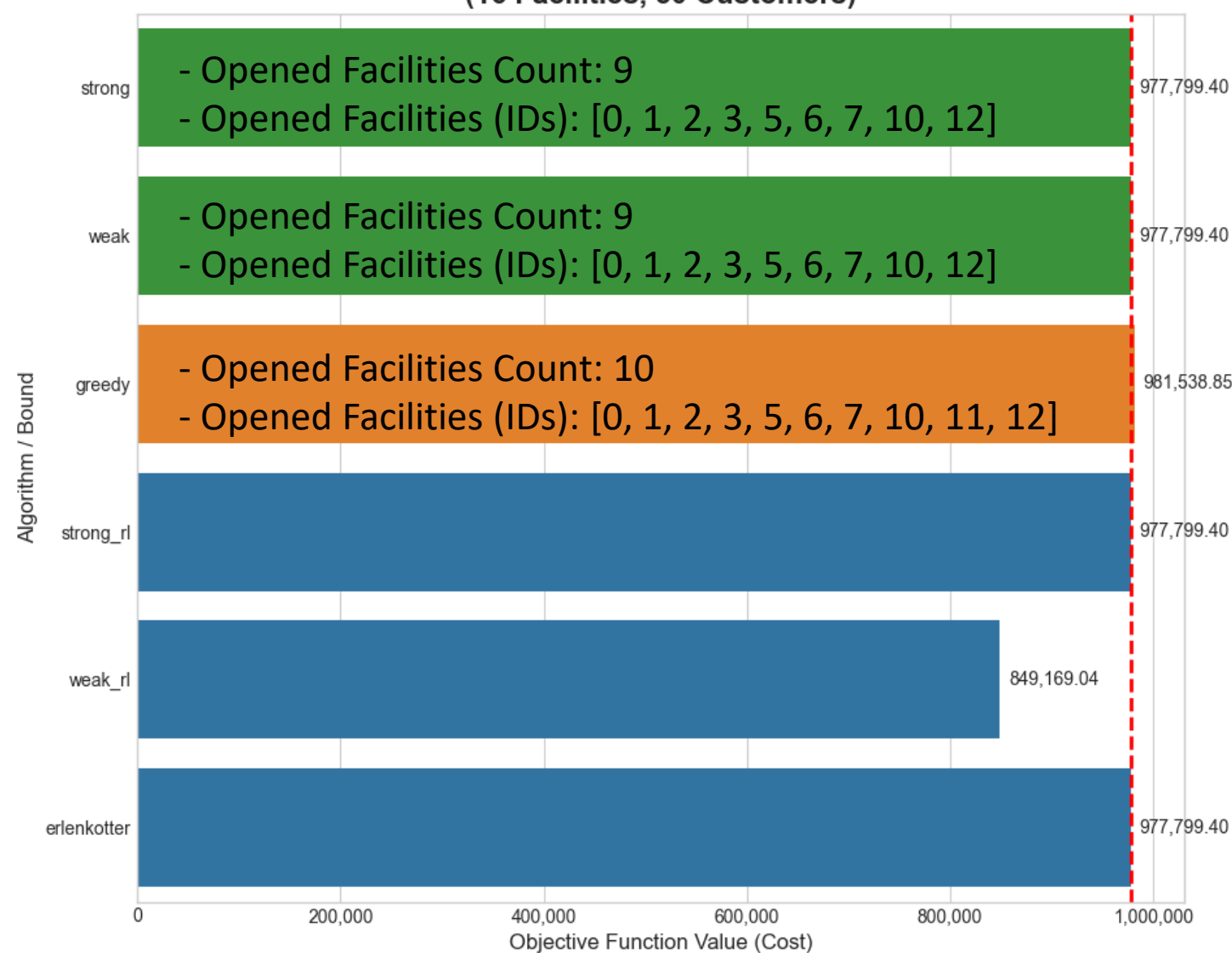
--- Algorithm: erlenkotter ---
- Objective Cost:          977799.4
- Computational Time (s):  0.0024
- Optimality Gap:          0.0%

--- Algorithm: greedy ---
- Objective Cost:          981538.85
- Computational Time (s):  0.0005
- Optimality Gap:          0.38%
- Opened Facilities Count: 10
- Opened Facilities (IDs): [0, 1, 2, 3, 5, 6, 7, 10, 11, 12]
```

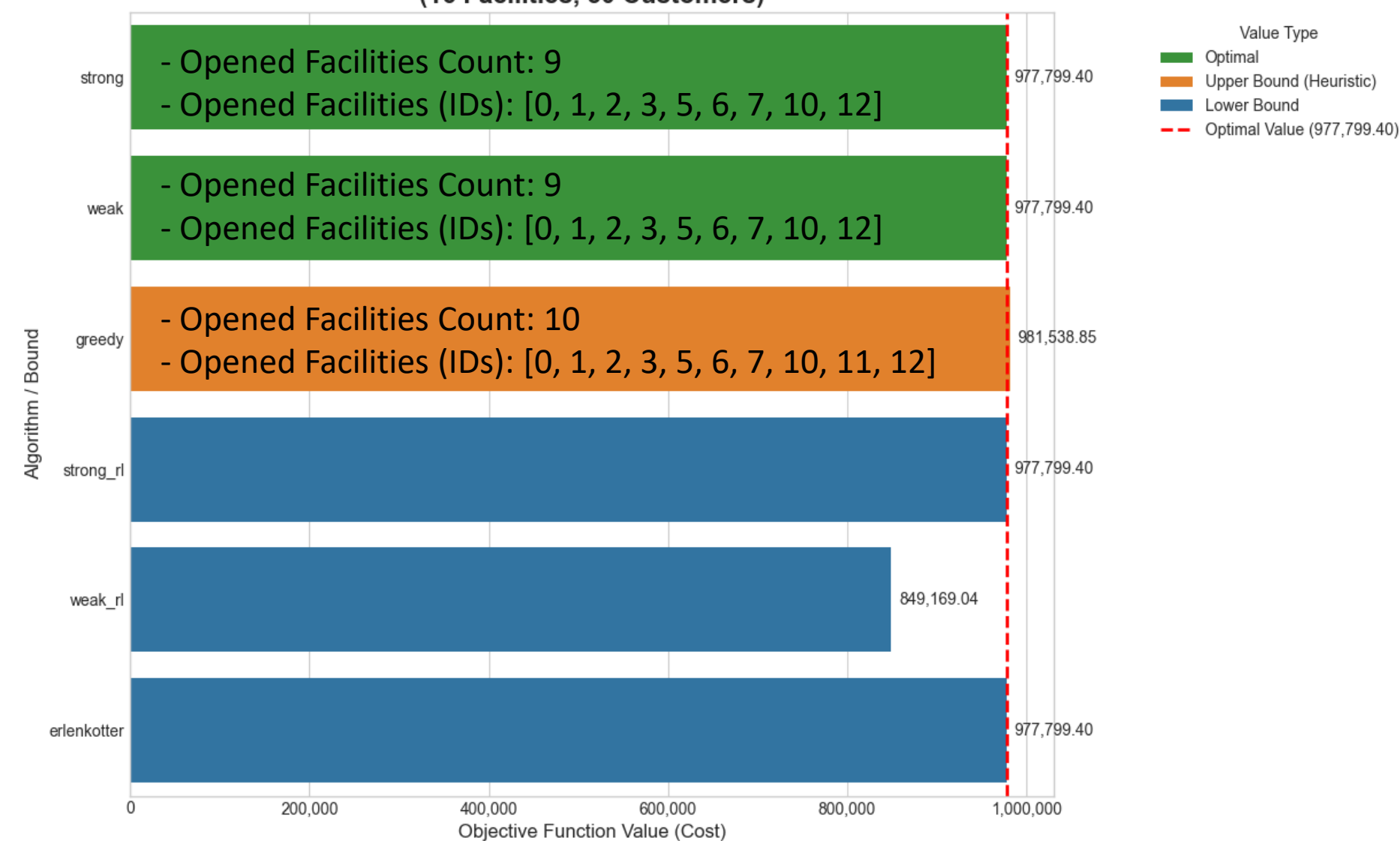
# Analisi di Performance per le singole istanze

## (Istanze small)

Performance Analysis for Instance: cap42.txt  
(16 Facilities, 50 Customers)



Performance Analysis for Instance: cap62.txt  
(16 Facilities, 50 Customers)

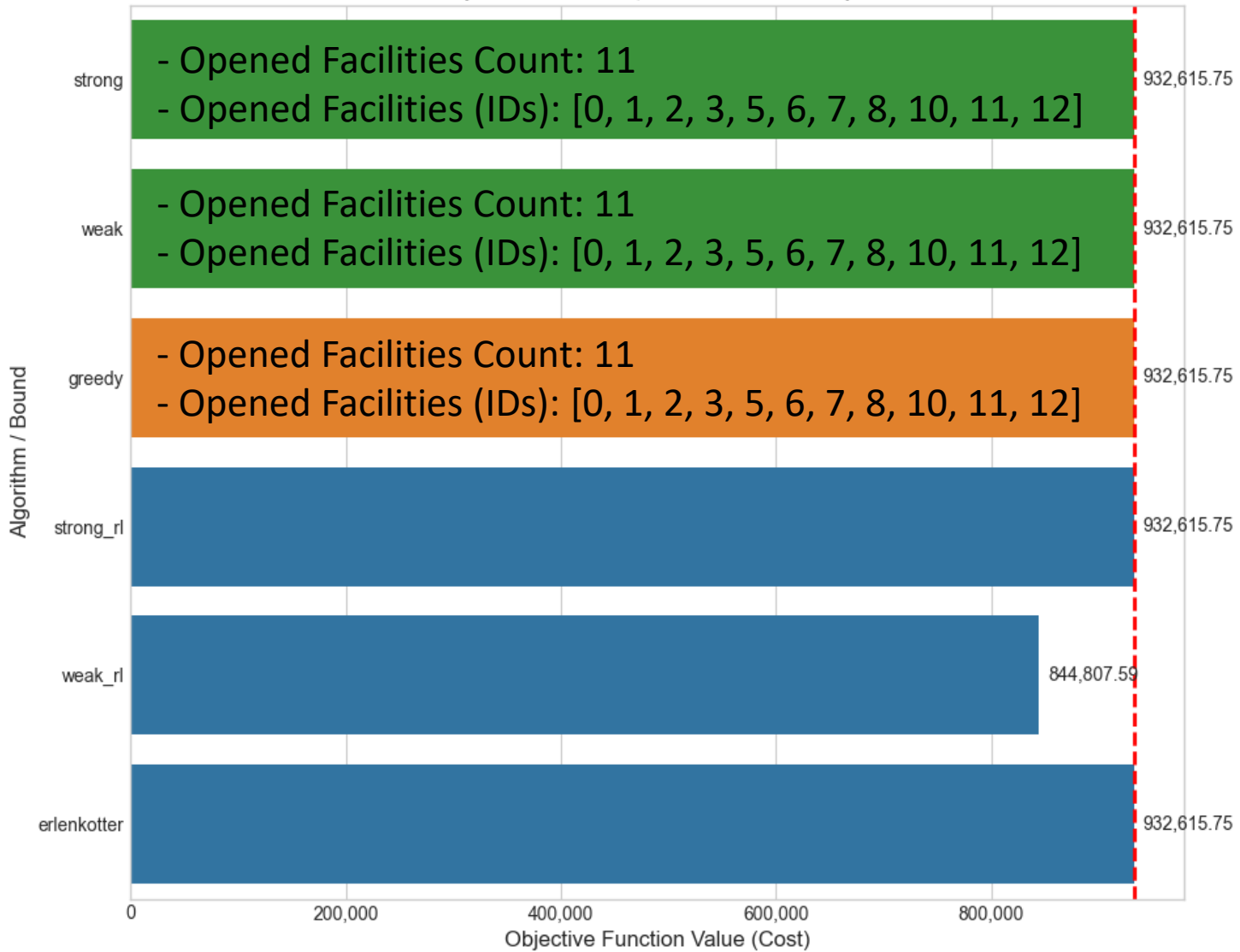




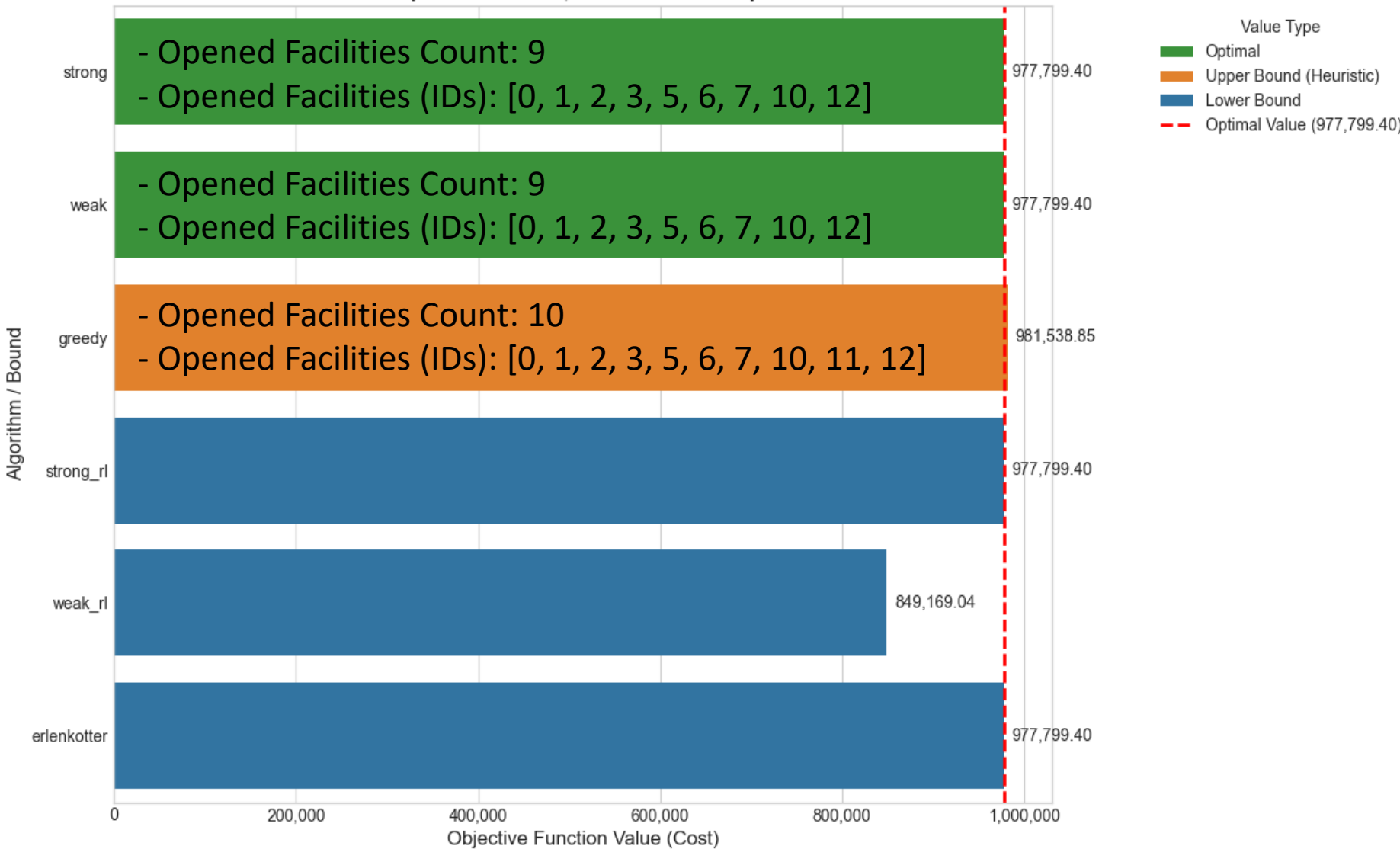
# Analisi di Performance per le singole istanze

## (Istanze small)

Performance Analysis for Instance: cap71.txt  
(16 Facilities, 50 Customers)



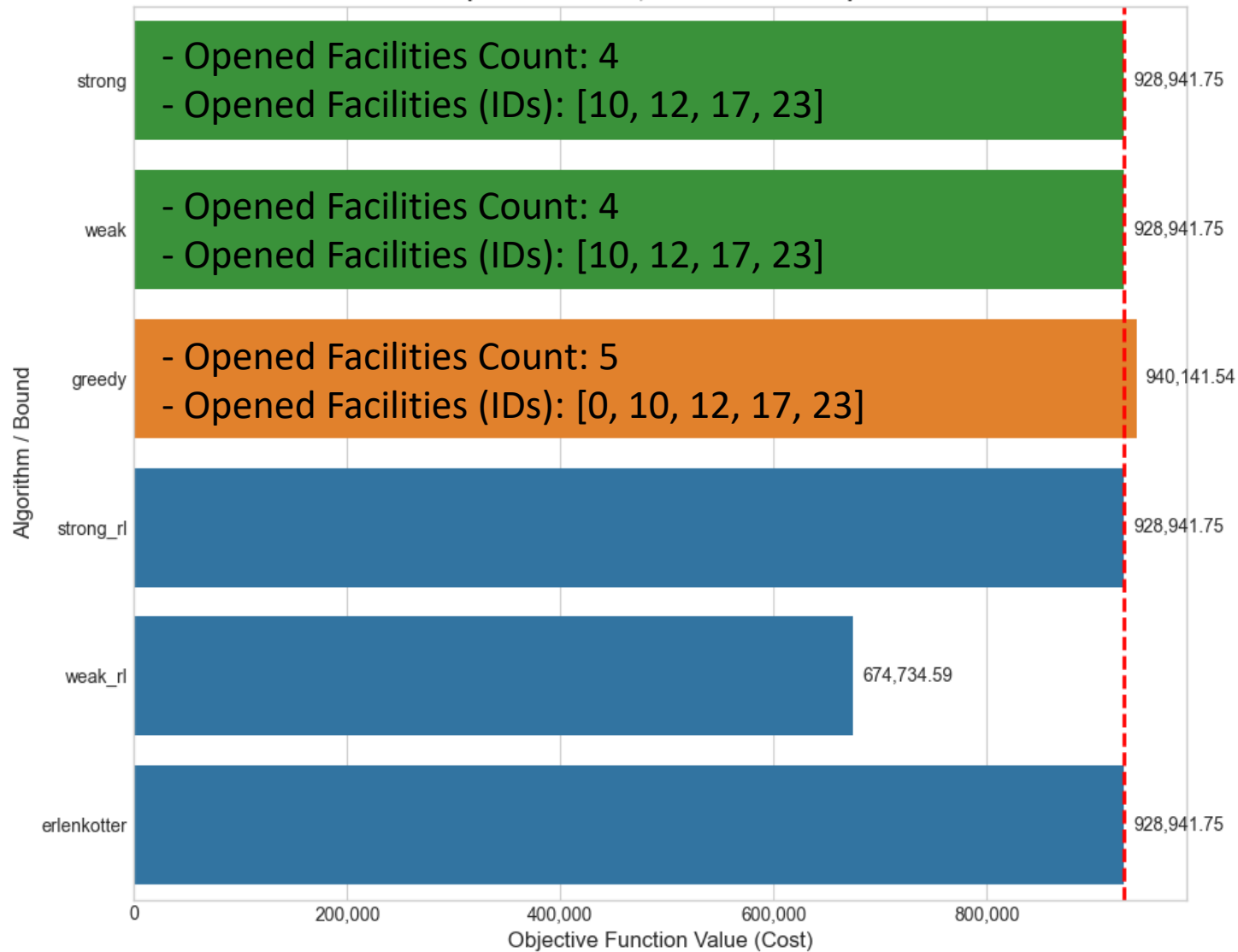
Performance Analysis for Instance: cap72.txt  
(16 Facilities, 50 Customers)



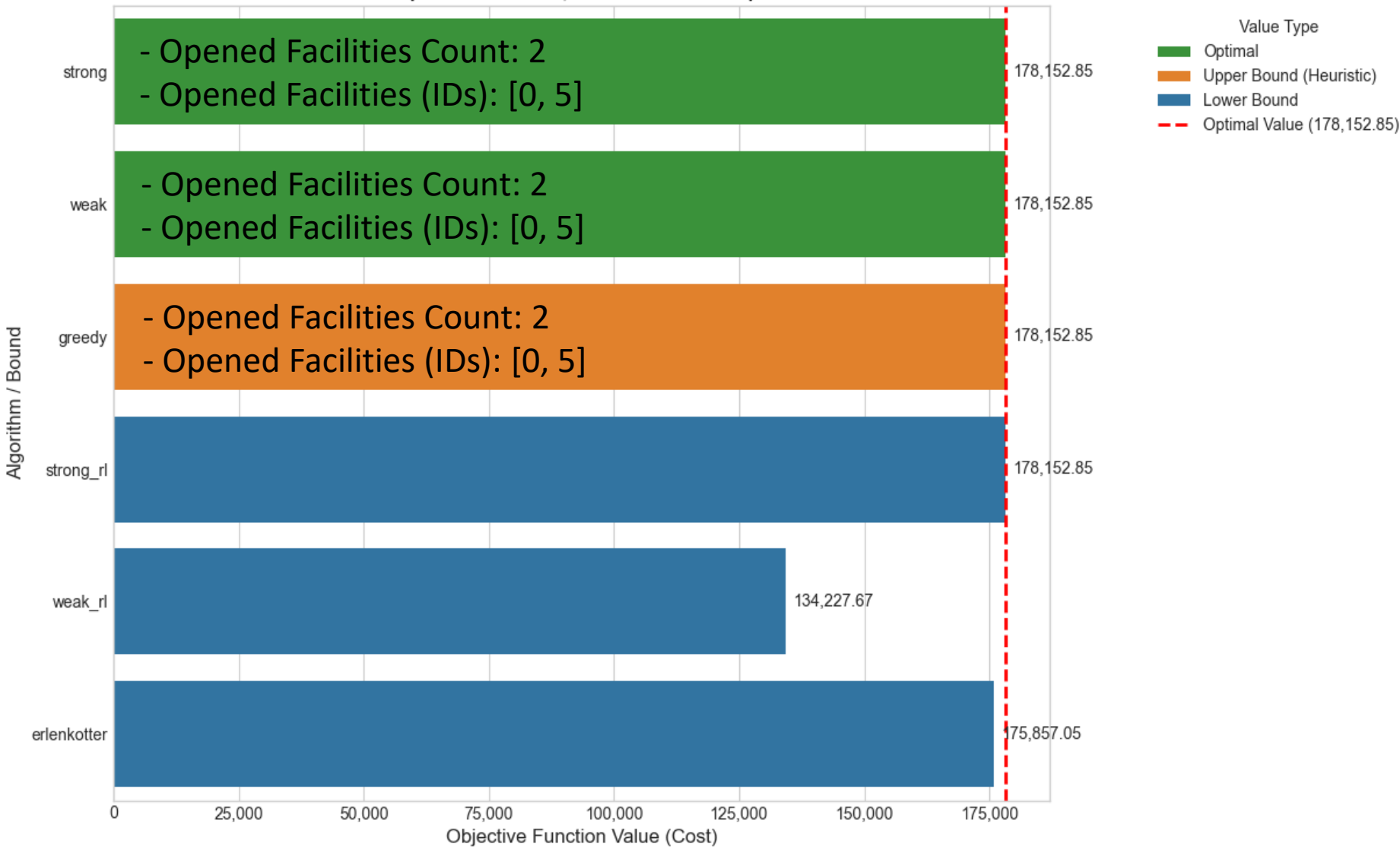
# Analisi di Performance per le singole istanze

## (Istanze small)

Performance Analysis for Instance: cap84.txt  
(25 Facilities, 50 Customers)

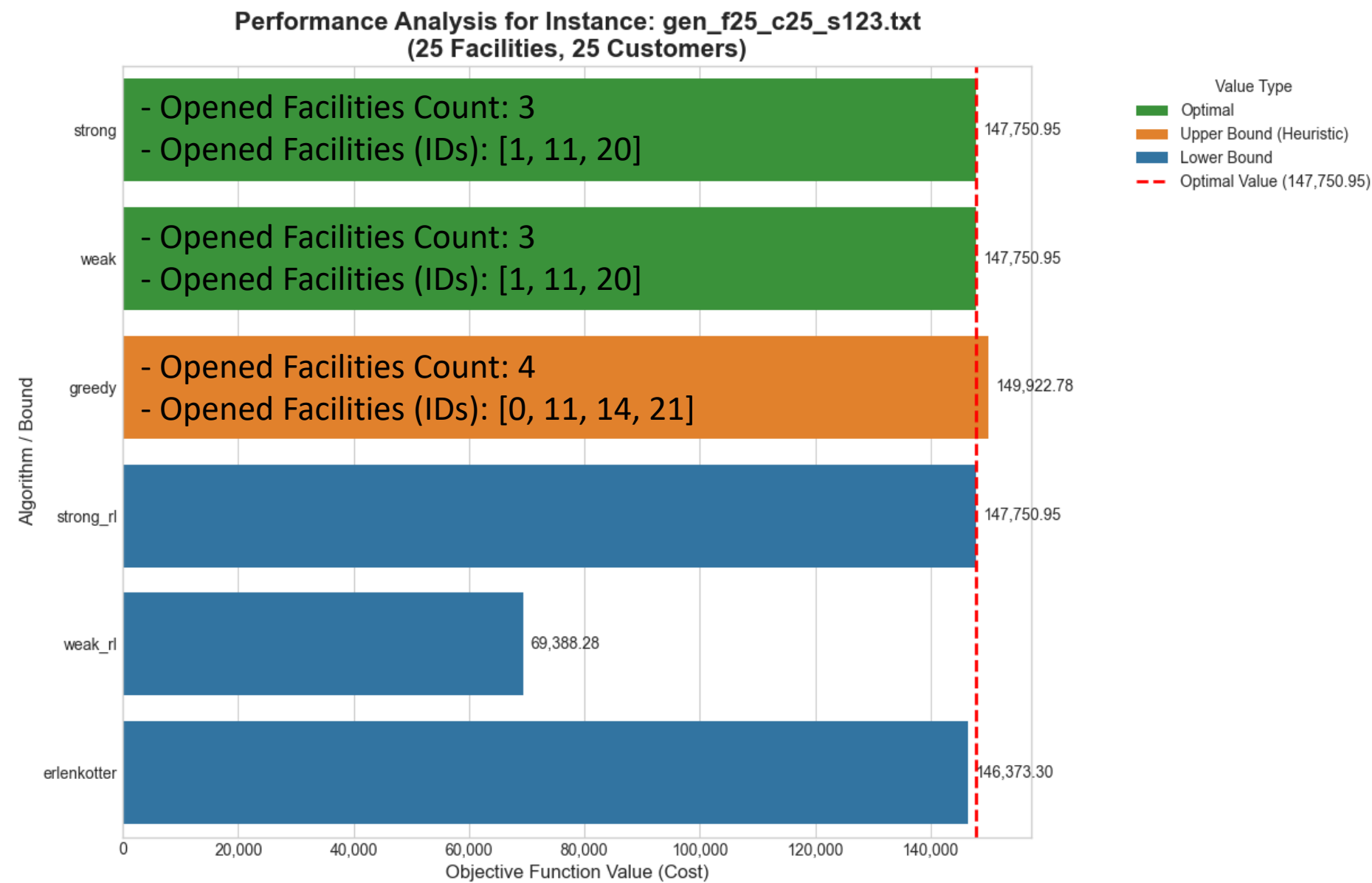


Performance Analysis for Instance: gen\_f10\_c30\_s123.txt  
(10 Facilities, 30 Customers)

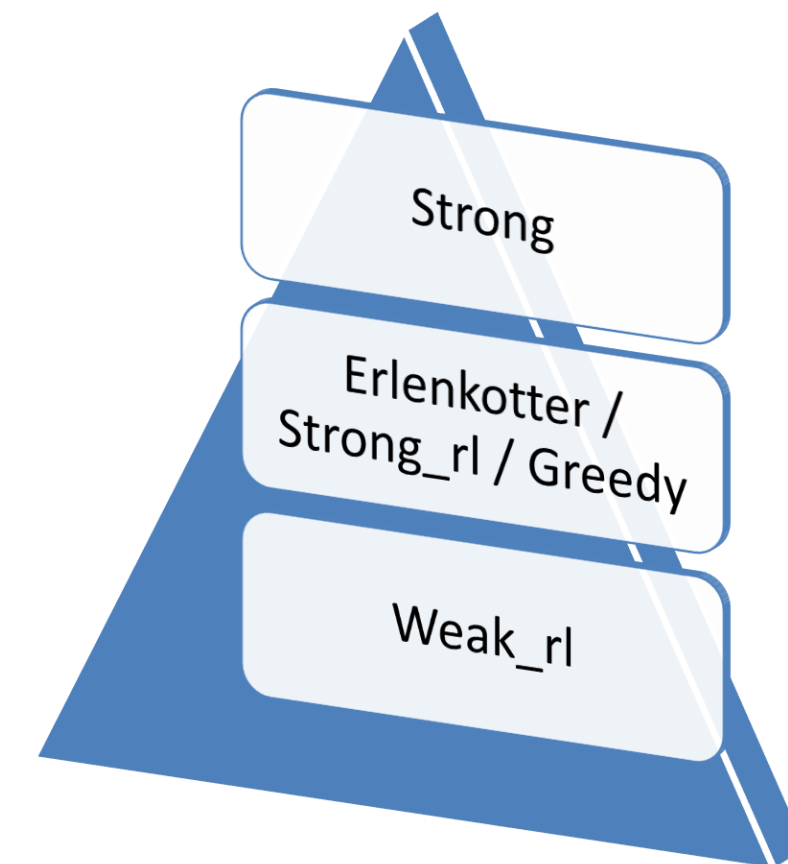


# Analisi di Performance per le singole istanze

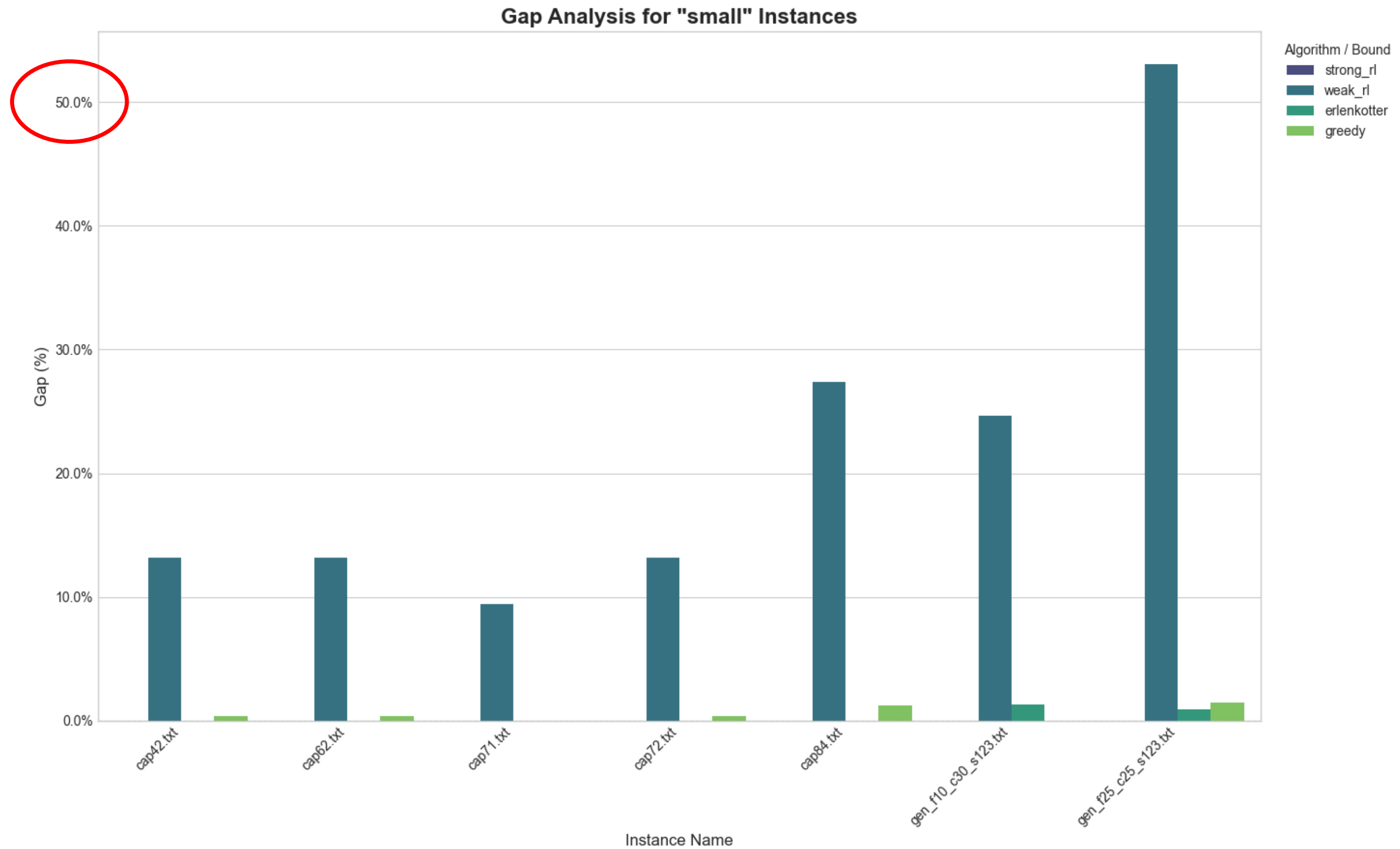
## (Istanze small)



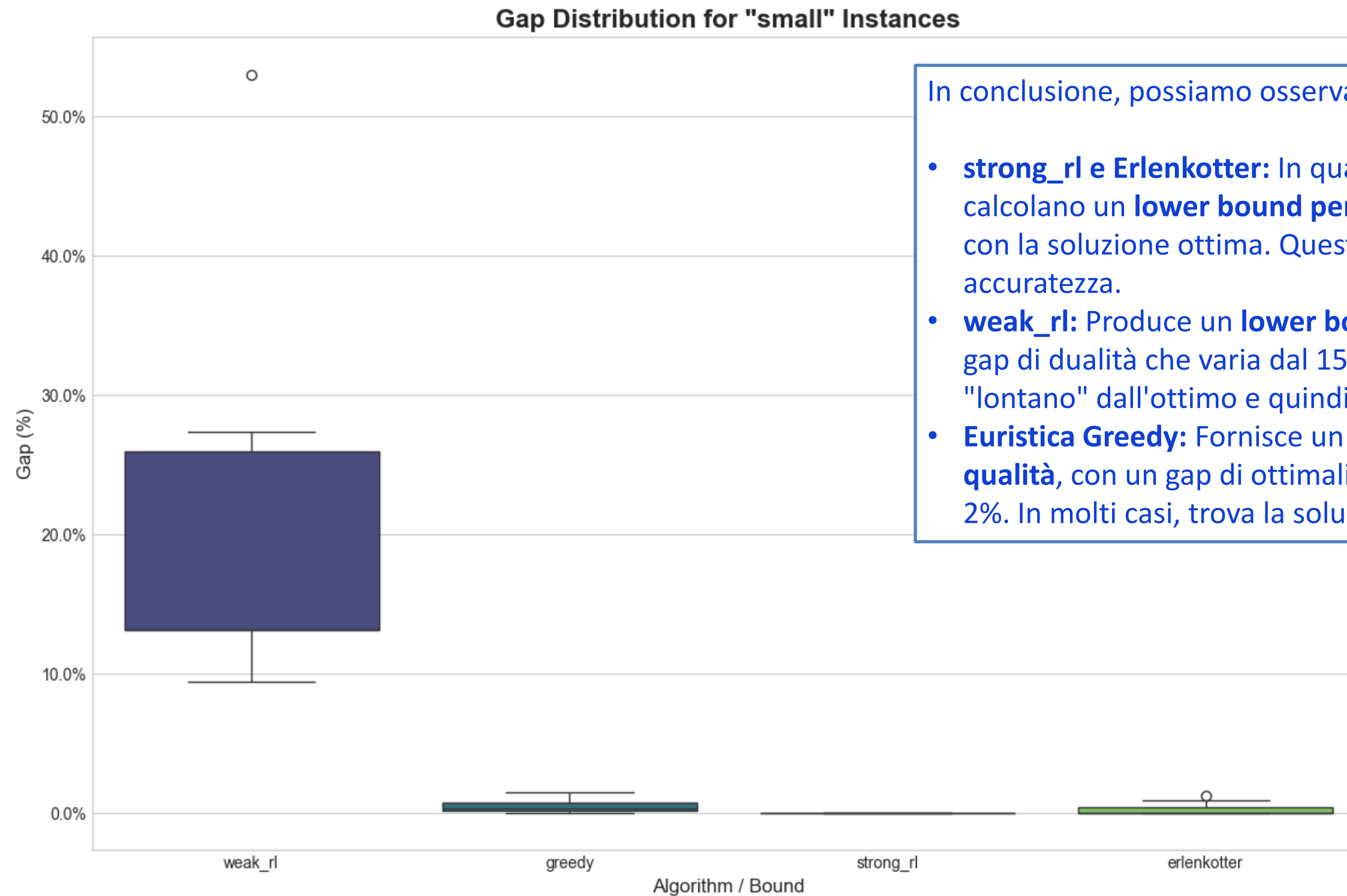
Gerarchia in base alle prestazioni:



# Analisi di Gap per le istanze small



# Distribuzione di Gap per le istanze small



In conclusione, possiamo osservare:

- **strong\_rl e Erlenkotter:** In quasi tutti i casi, entrambi calcolano un **lower bound perfetto**, il cui valore coincide con la soluzione ottima. Questo dimostra la loro eccezionale accuratezza.
- **weak\_rl:** Produce un **lower bound di scarsa qualità**, con un gap di dualità che varia dal 15% al 30%. È un limite molto "lontano" dall'ottimo e quindi poco informativo.
- **Euristica Greedy:** Fornisce un **upper bound di buona qualità**, con un gap di ottimalità quasi sempre inferiore al 2%. In molti casi, trova la soluzione ottima.

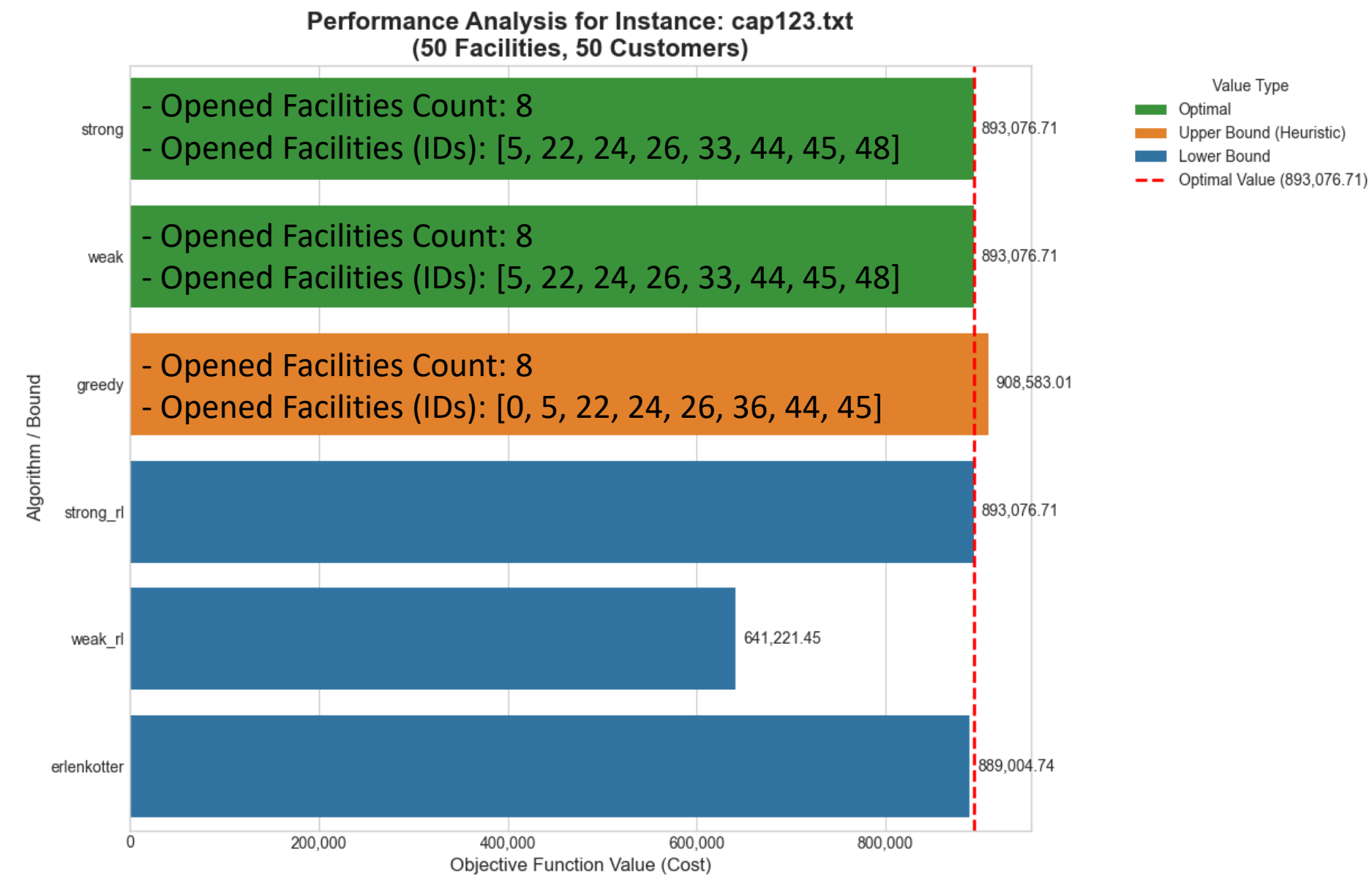
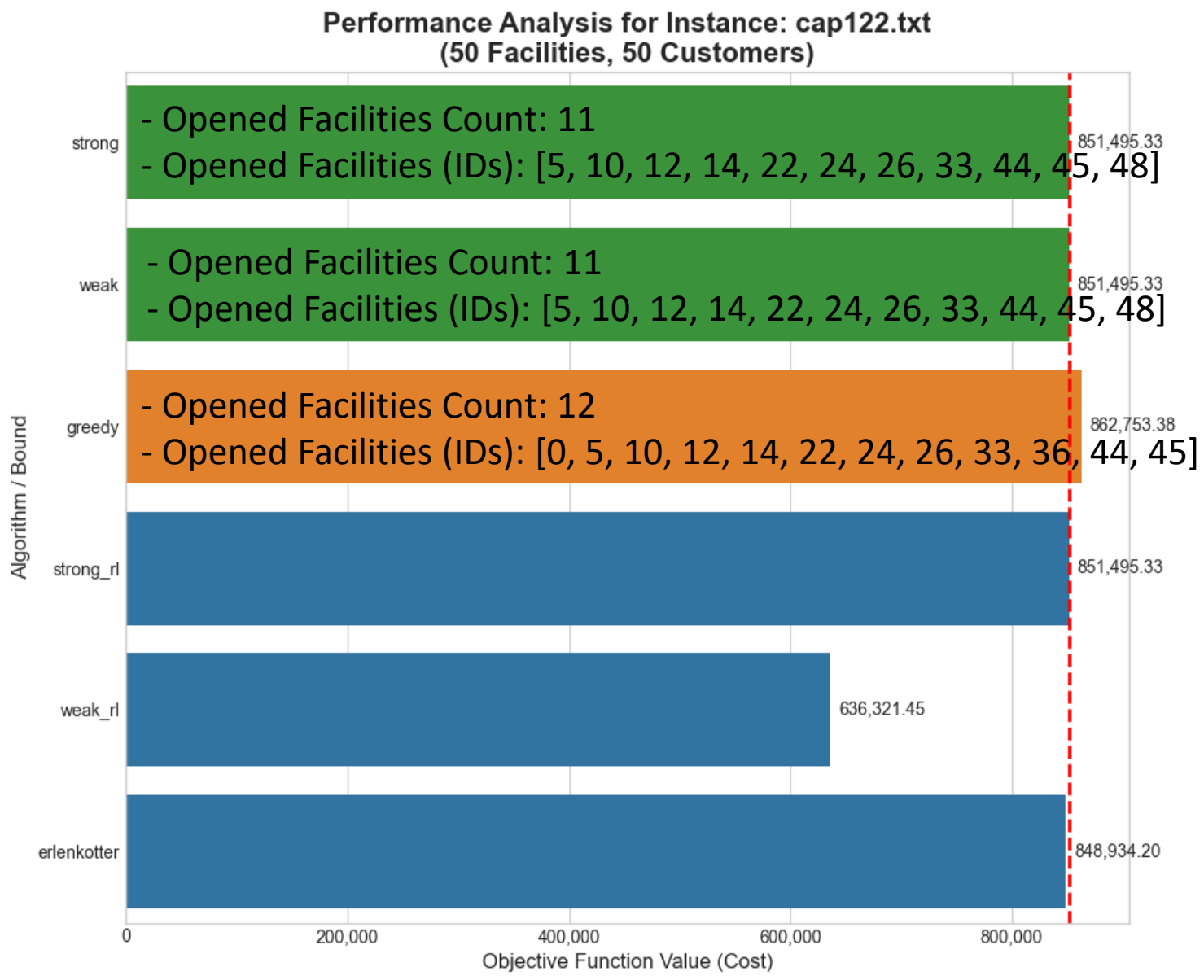


# Analisi di Performance per le singole istanze medio



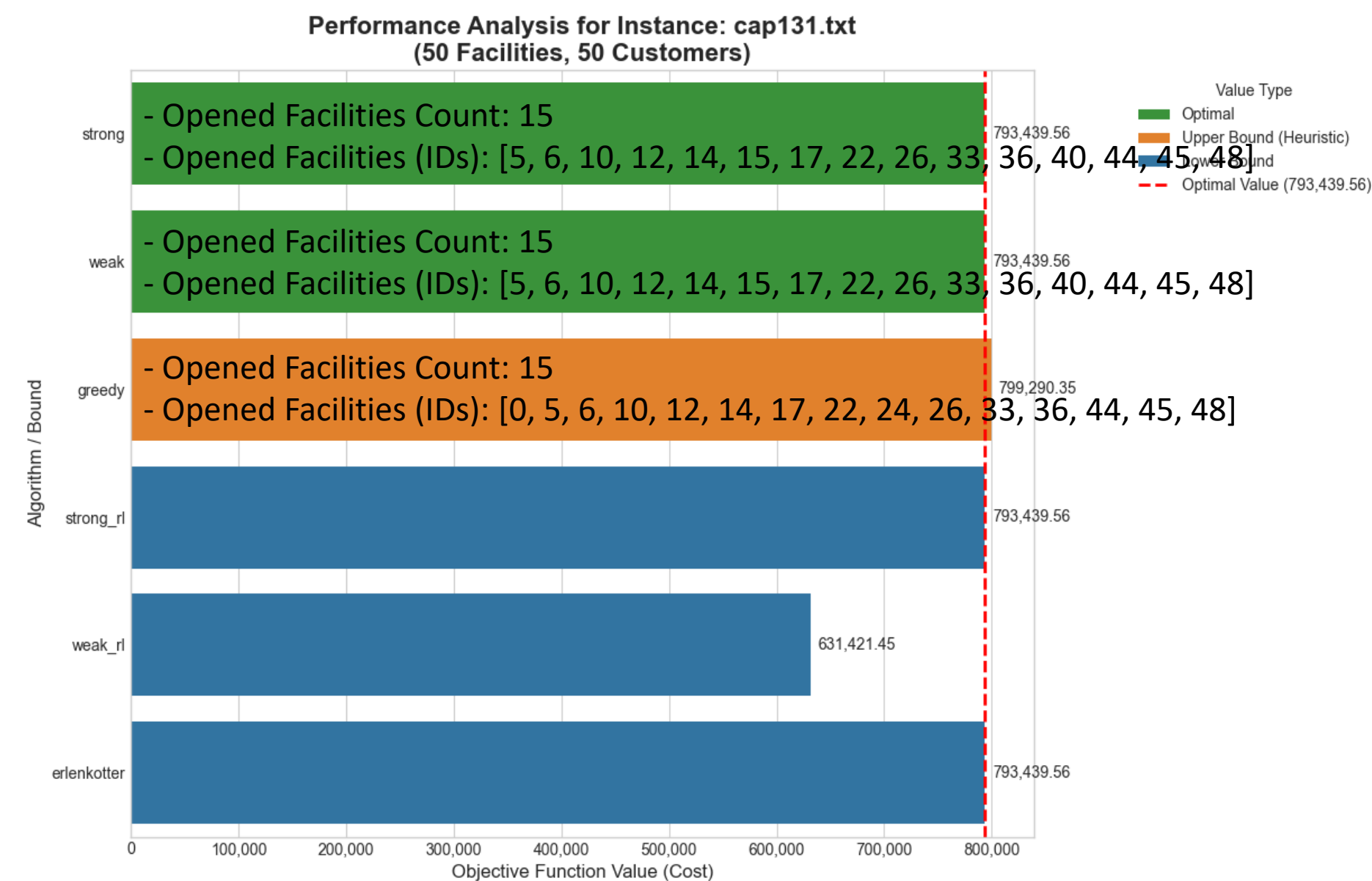
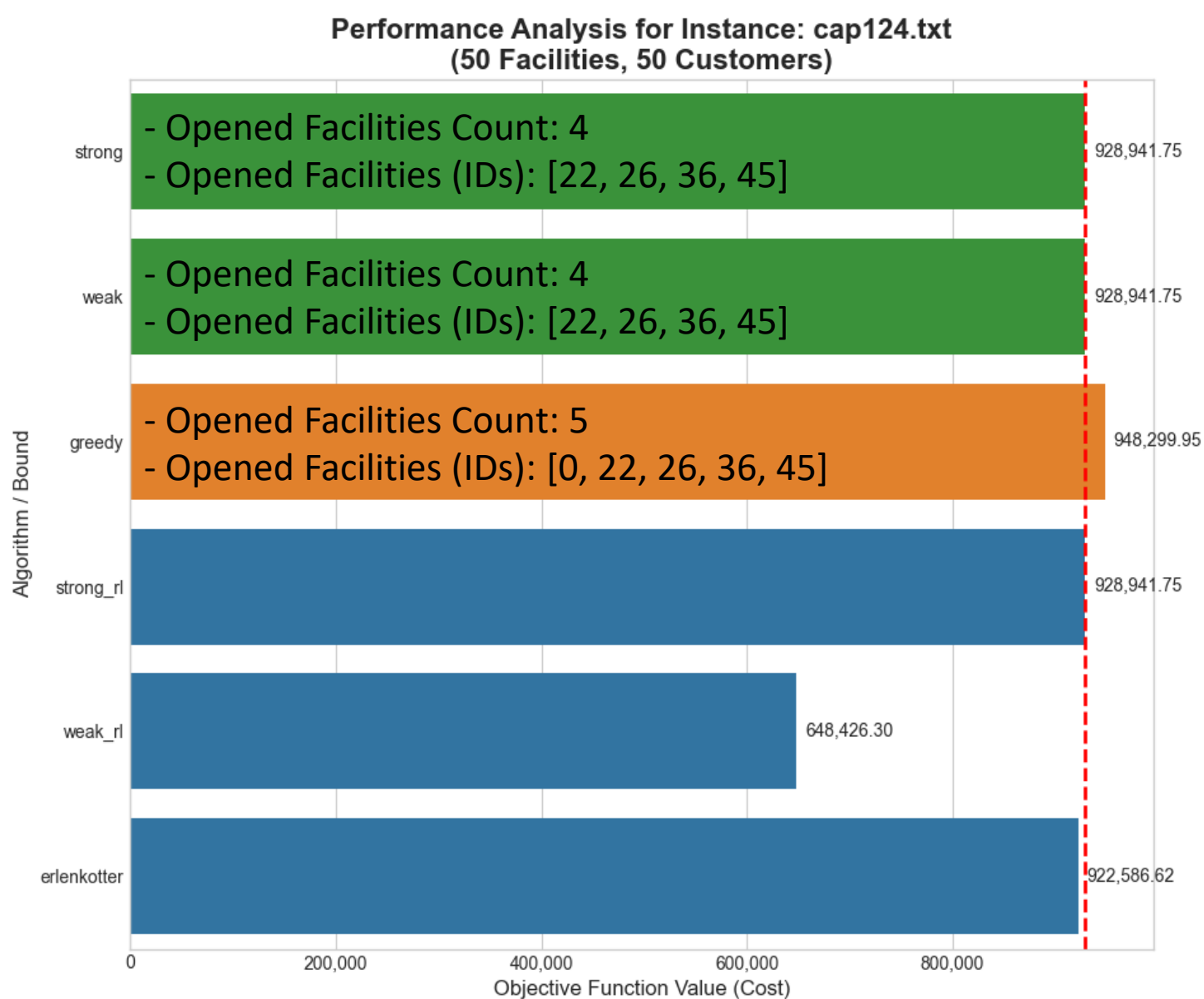
# Analisi di Performance per le singole istanze

## (Istanze medio)



# Analisi di Performance per le singole istanze

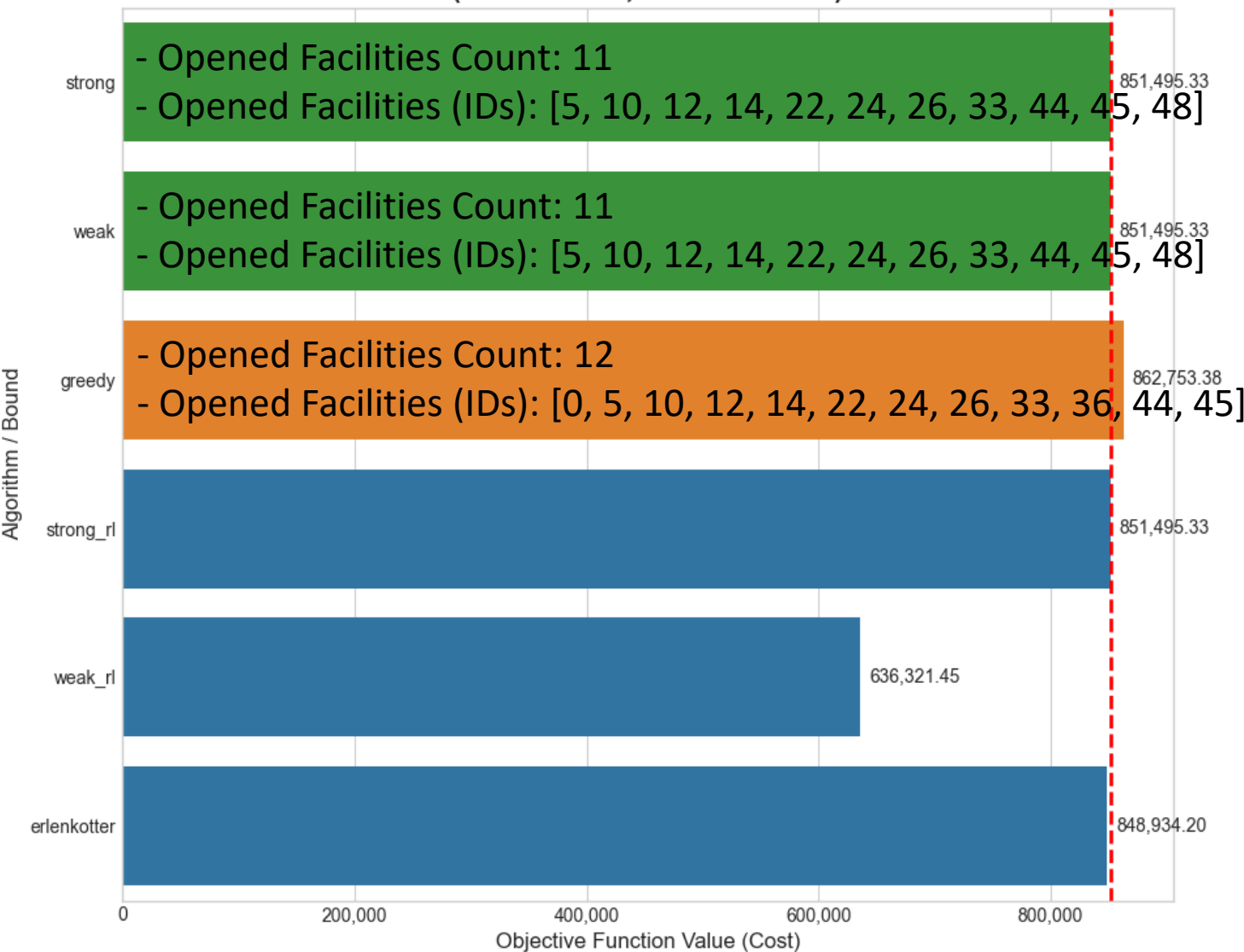
## (Istanze medio)



# Analisi di Performance per le singole istanze

## (Istanze medio)

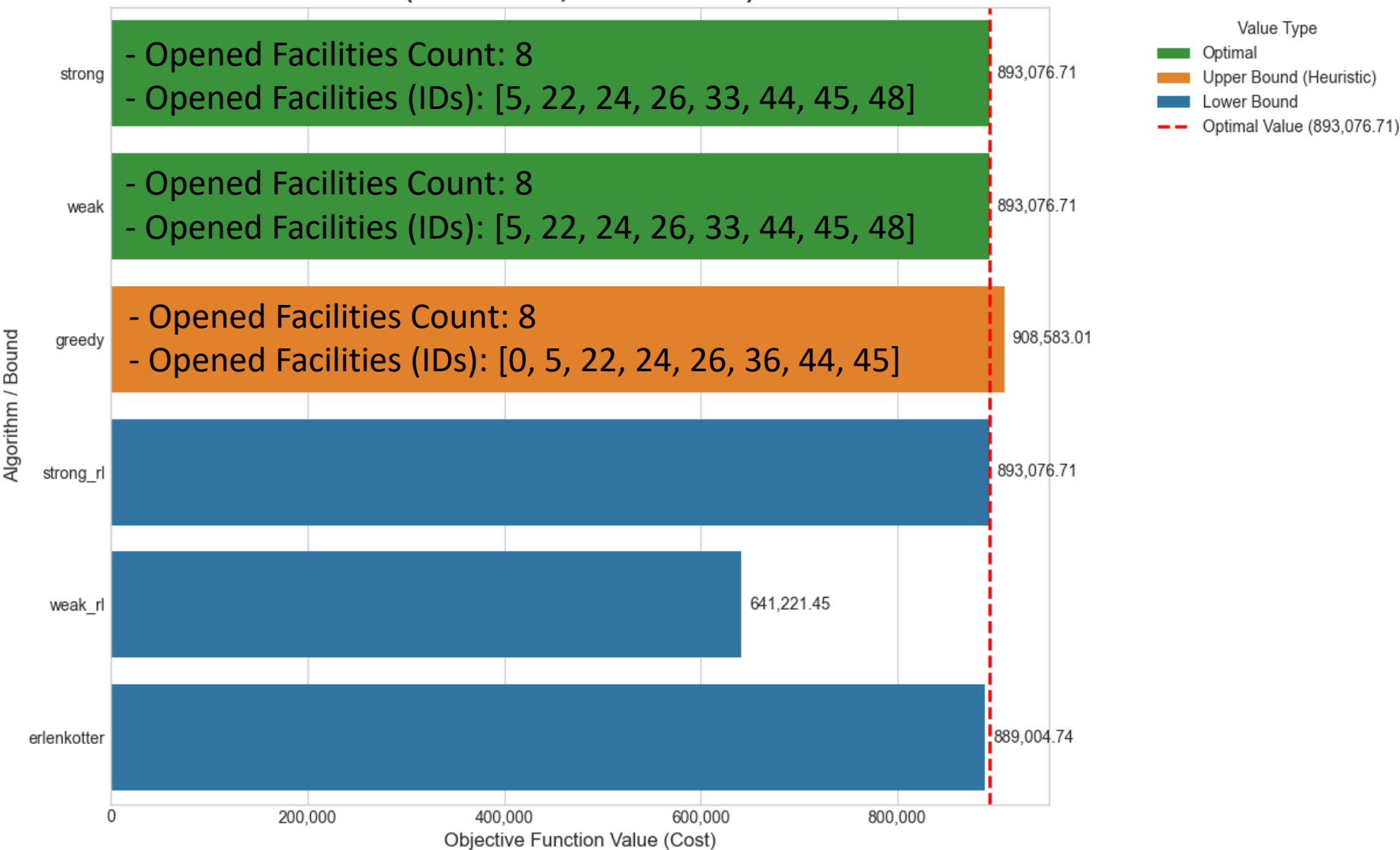
Performance Analysis for Instance: cap132.txt  
(50 Facilities, 50 Customers)



Value Type

- Optimal
- Upper Bound (Heuristic)
- Lower Bound
- Optimal Value (851,495.33)

Performance Analysis for Instance: cap133.txt  
(50 Facilities, 50 Customers)

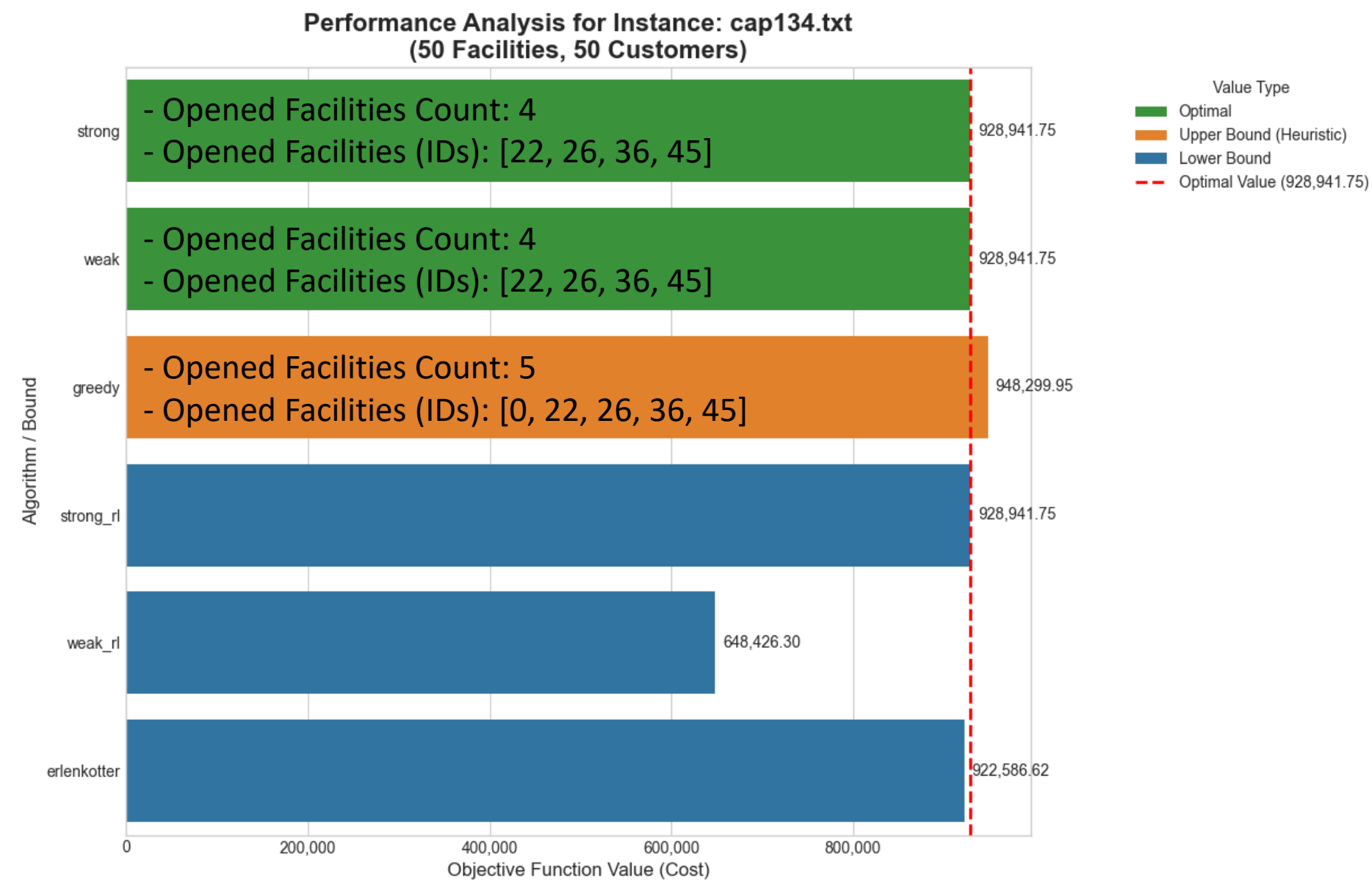


Value Type

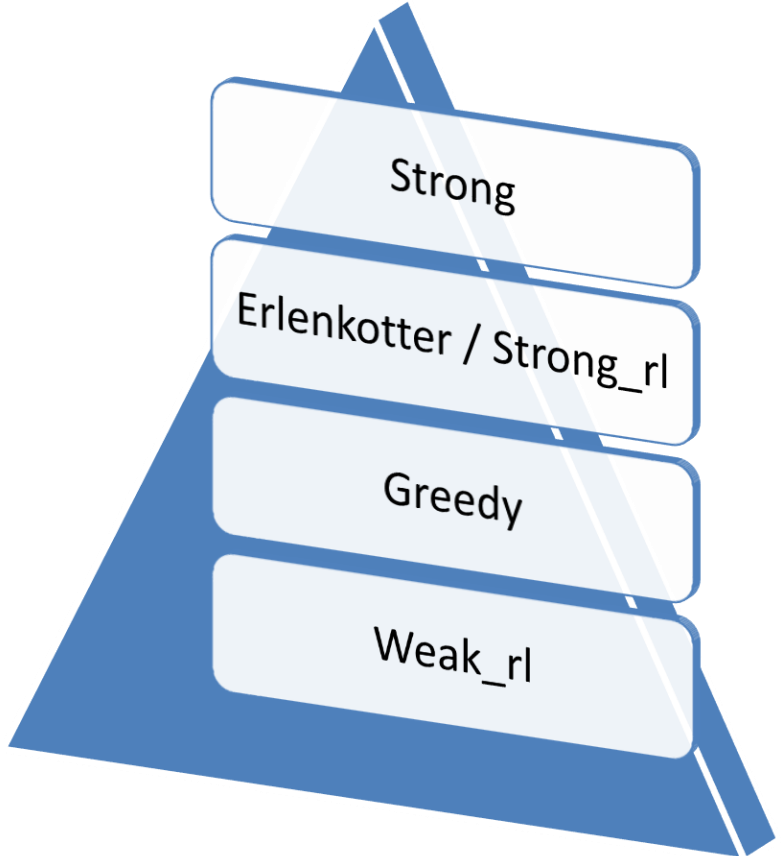
- Optimal
- Upper Bound (Heuristic)
- Lower Bound
- Optimal Value (893,076.71)

# Analisi di Performance per le singole istanze

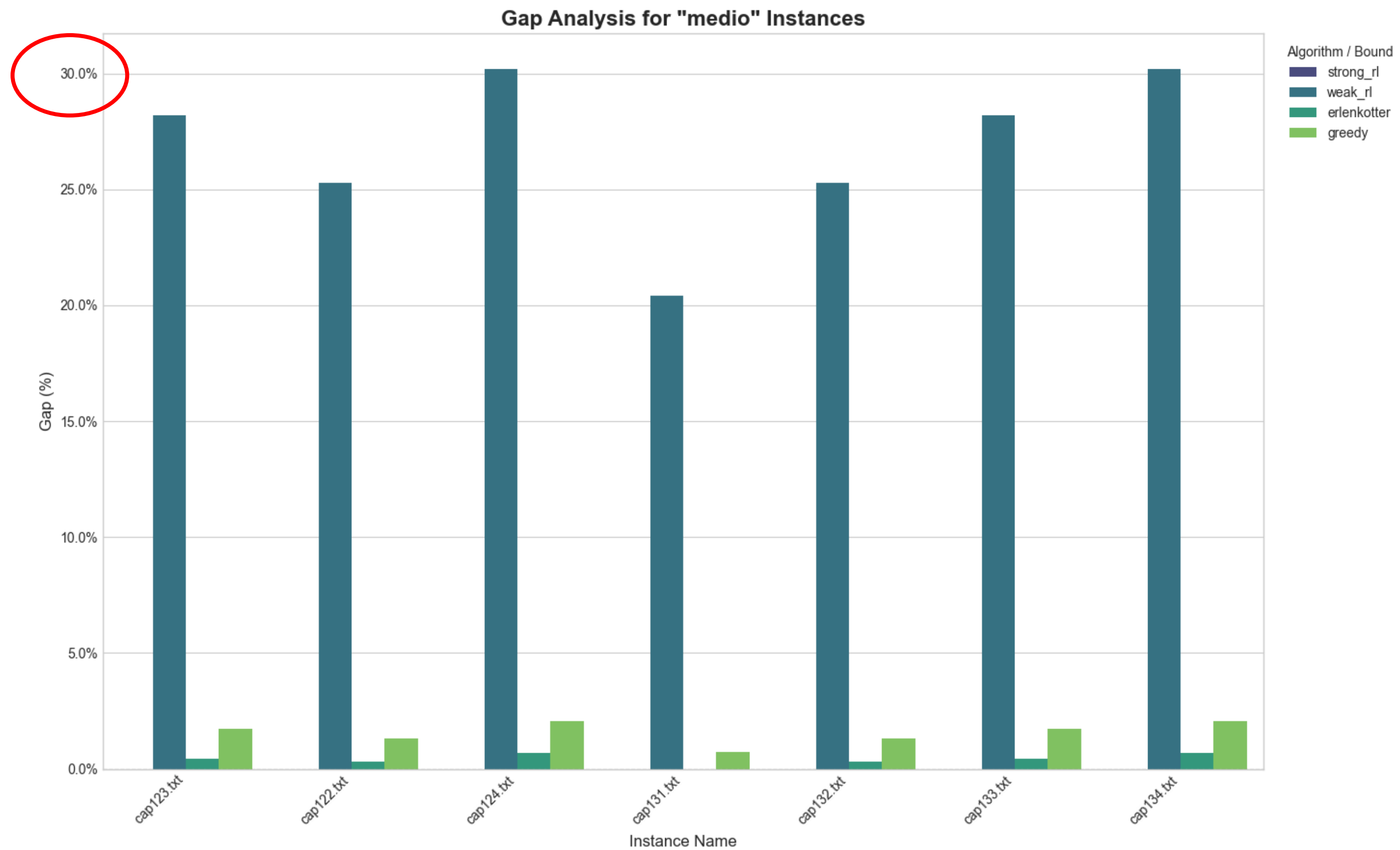
## (Istanze medio)



Gerarchia in base alle prestazioni:

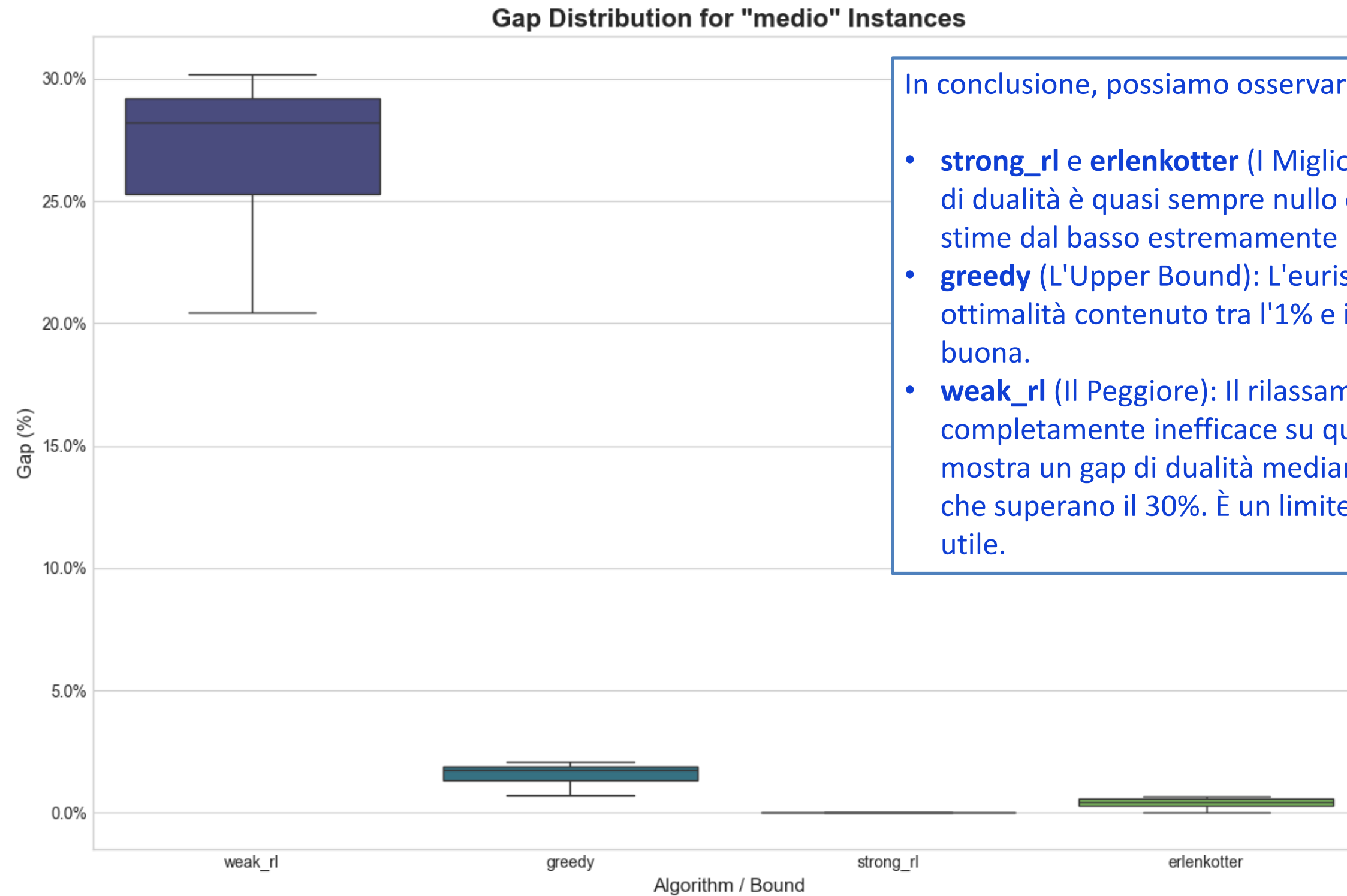


# Analisi di Gap per le istanze medio





# Distribuzione di Gap per le istanze medio



In conclusione, possiamo osservare:

- **strong\_rl** e **erlenkotter** (I Migliori Lower Bounds): Il loro gap di dualità è quasi sempre nullo o inferiore all'1%. Forniscono stime dal basso estremamente accurate e affidabili.
- **greedy** (L'Upper Bound): L'euristica greedy ha un gap di ottimalità contenuto tra l'1% e il 2.5%. La sua qualità è buona.
- **weak\_rl** (Il Peggior): Il rilassamento debole si dimostra completamente inefficace su queste istanze. Il box plot mostra un gap di dualità mediano di circa il 28%, con valori che superano il 30%. È un limite troppo "lontano" per essere utile.

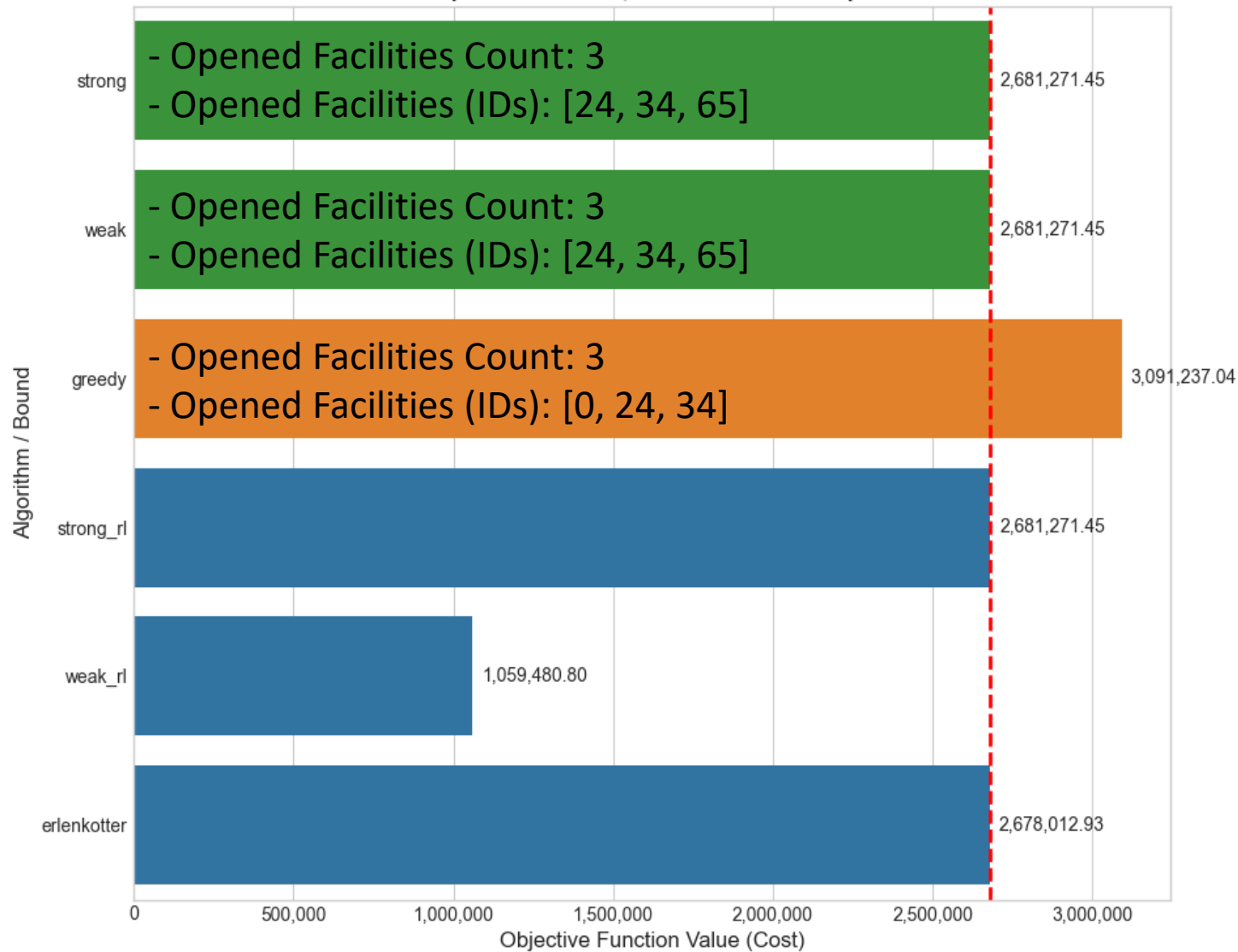


# Analisi di Performance per le singole istanze big

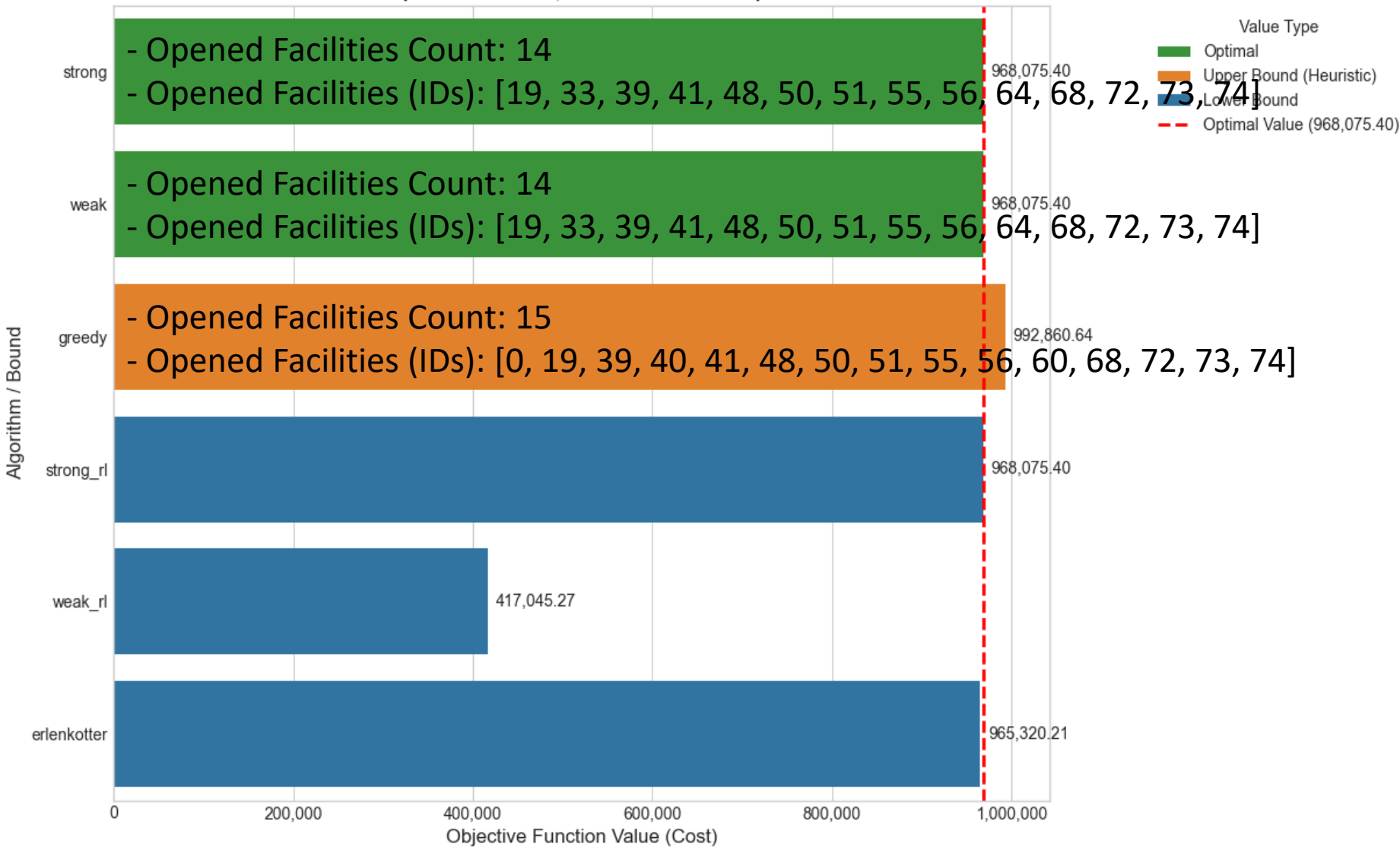
# Analisi di Performance per le singole istanze

## (Istanze big)

Performance Analysis for Instance: gen\_f75\_c150\_s123.txt  
(75 Facilities, 150 Customers)

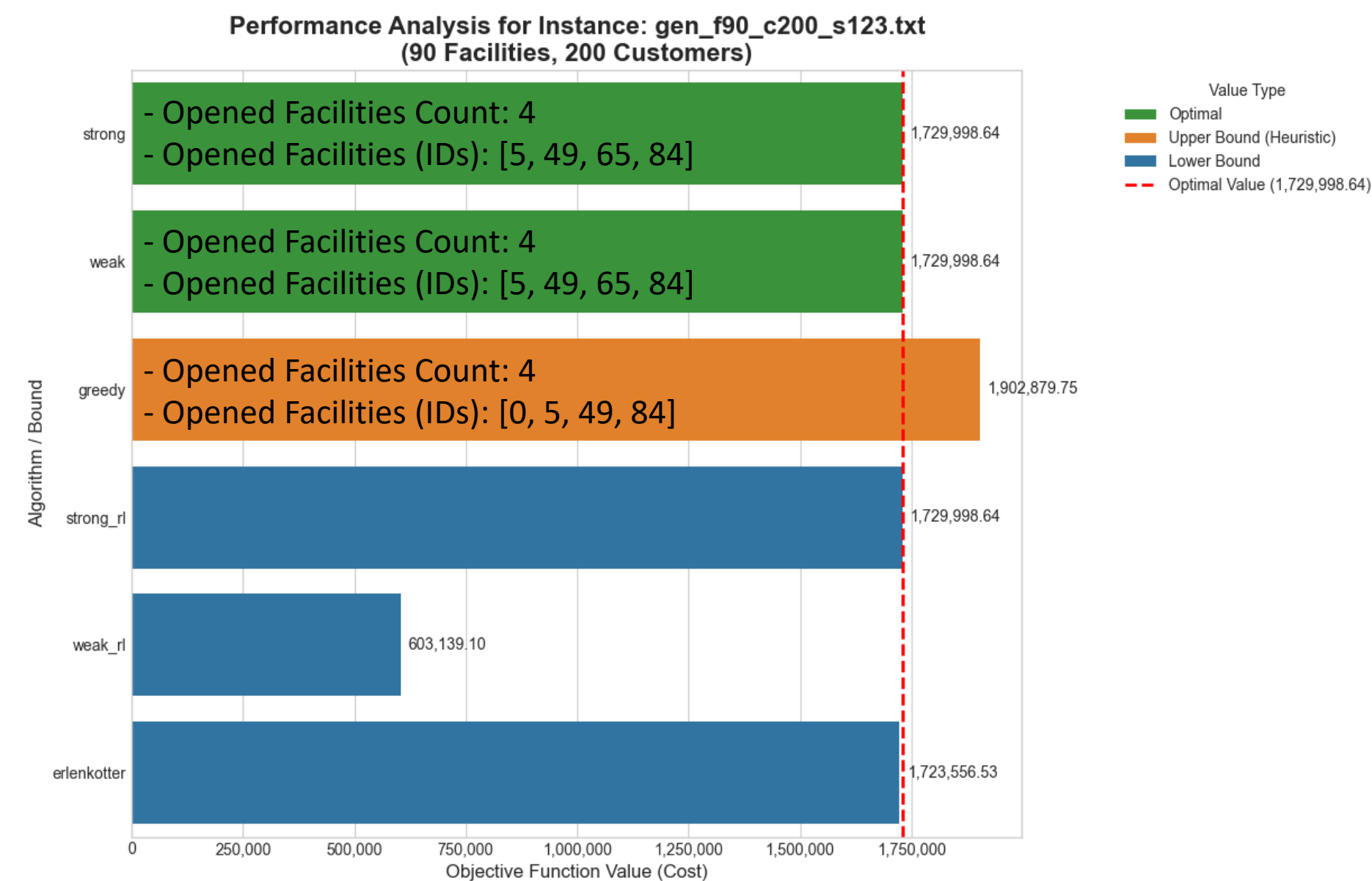
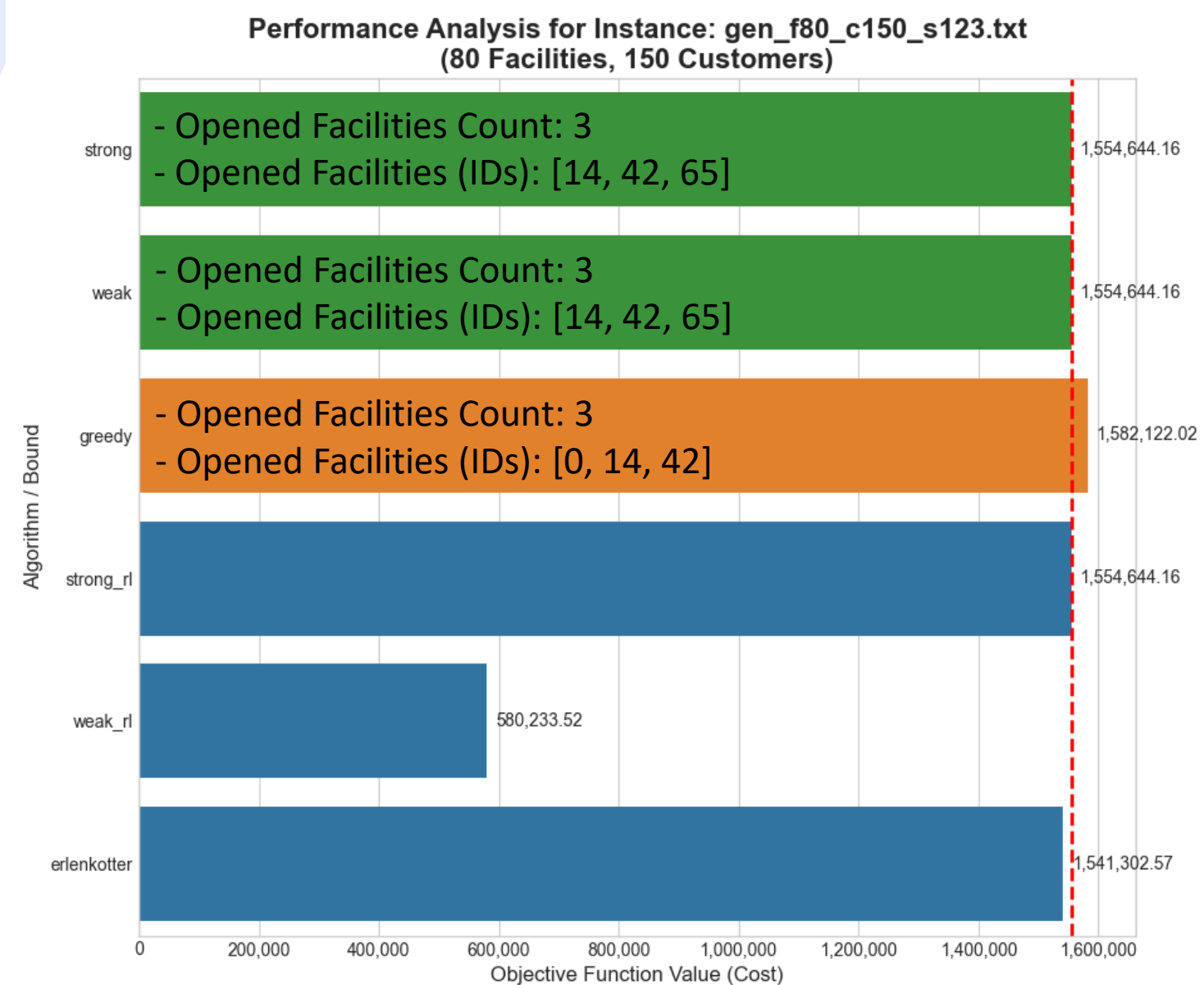


Performance Analysis for Instance: gen\_f75\_c200\_s123.txt  
(75 Facilities, 200 Customers)



# Analisi di Performance per le singole istanze

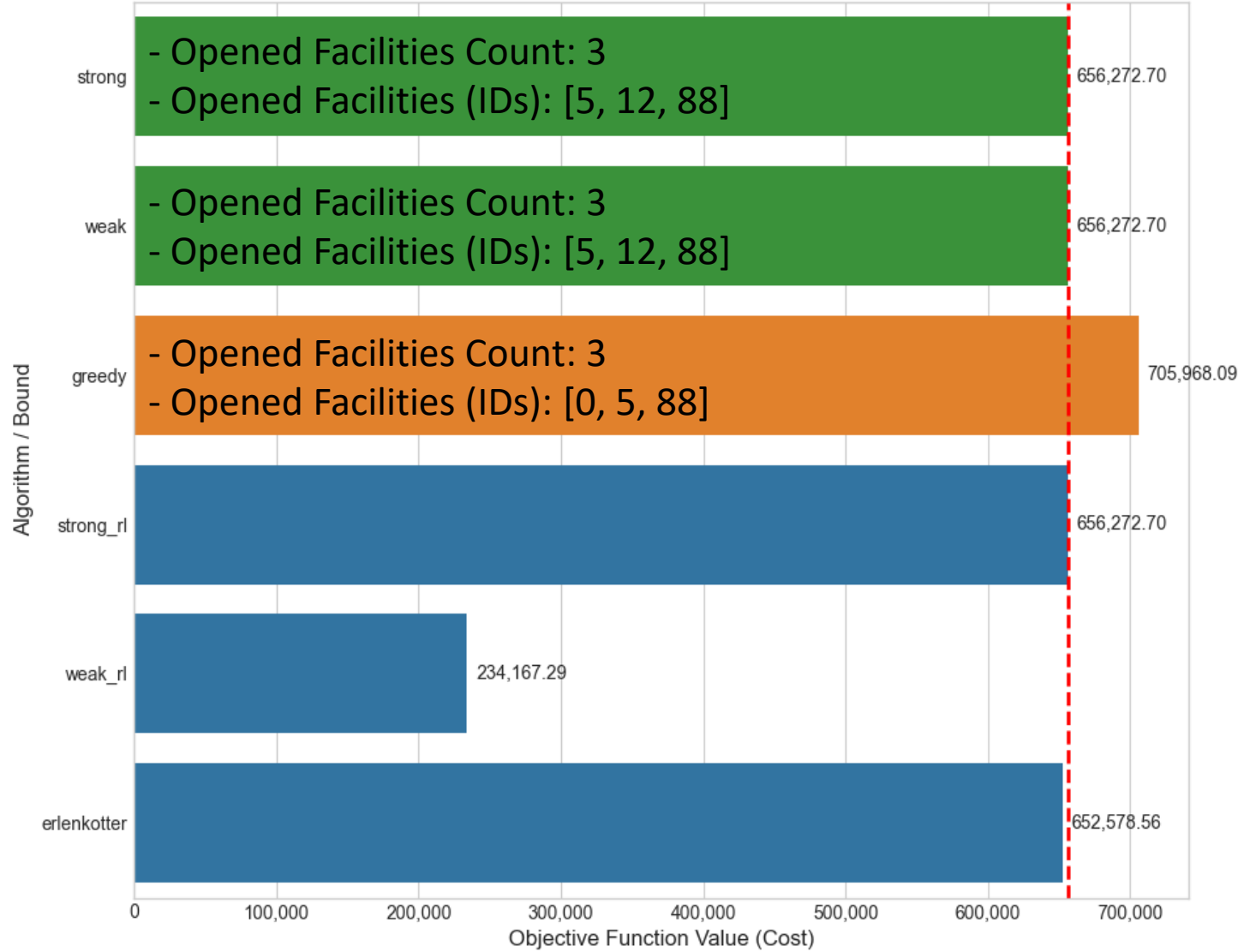
## (Istanze big)



# Analisi di Performance per le singole istanze

## (Istanze big)

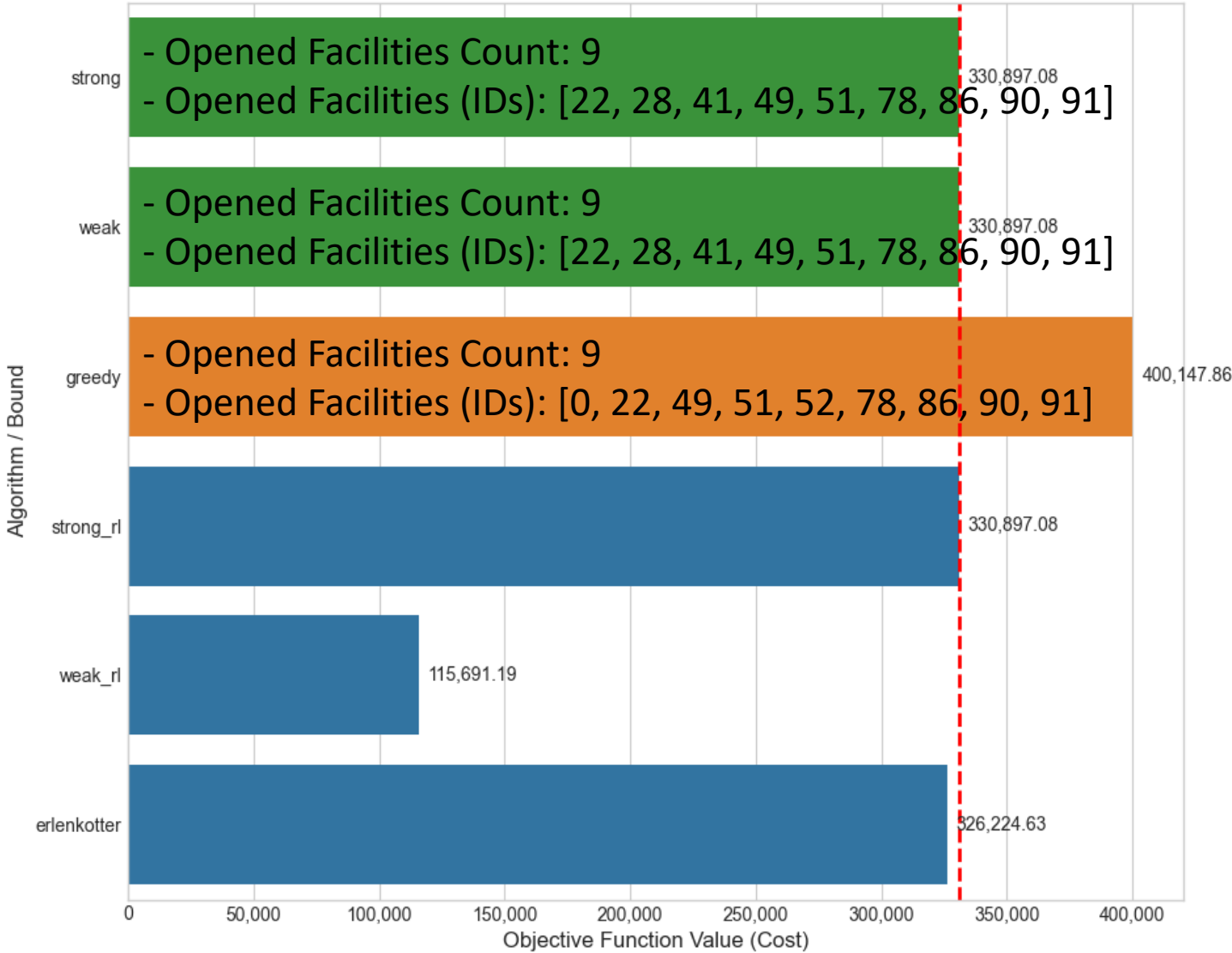
Performance Analysis for Instance: gen\_f90\_c250\_s123.txt  
(90 Facilities, 250 Customers)



Value Type

- Optimal
- Upper Bound (Heuristic)
- Lower Bound
- Optimal Value (656,272.70)

Performance Analysis for Instance: gen\_f100\_c150\_s123.txt  
(100 Facilities, 150 Customers)

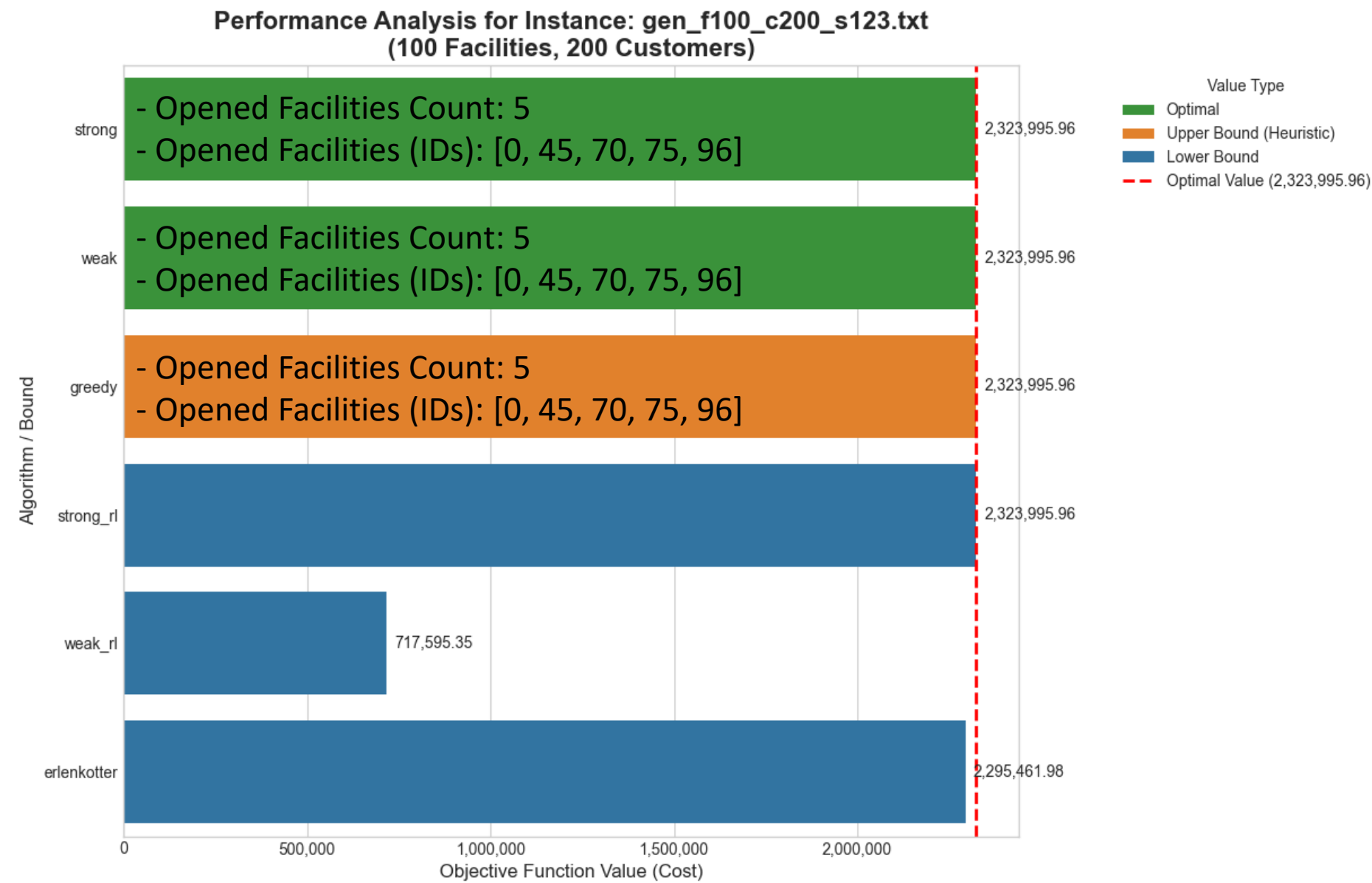


Value Type

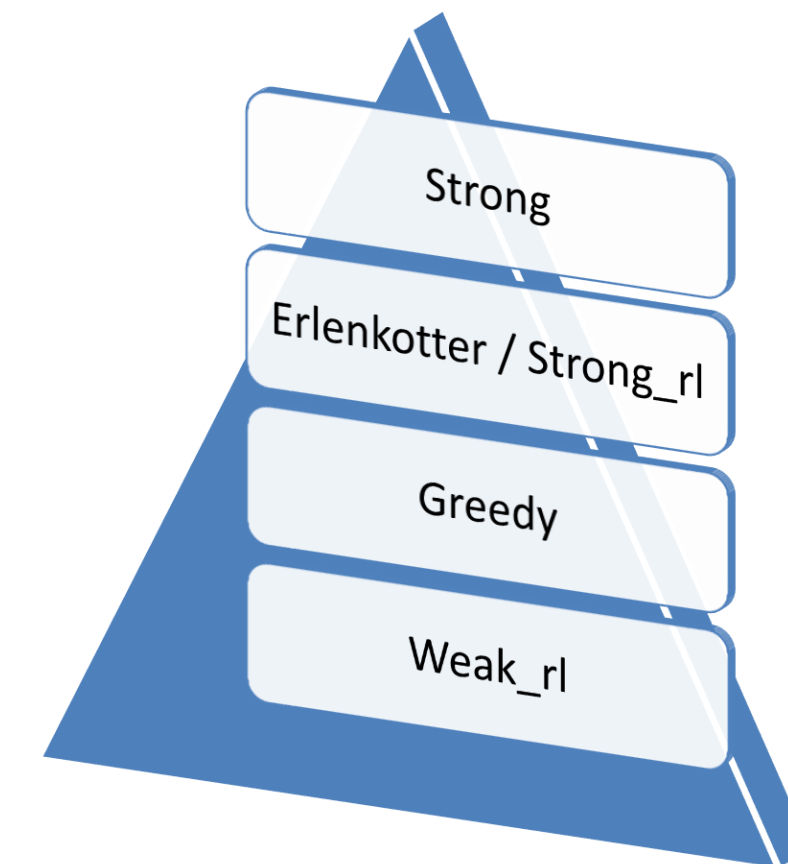
- Optimal
- Upper Bound (Heuristic)
- Lower Bound
- Optimal Value (330,897.08)

# Analisi di Performance per le singole istanze

## (Istanze big)

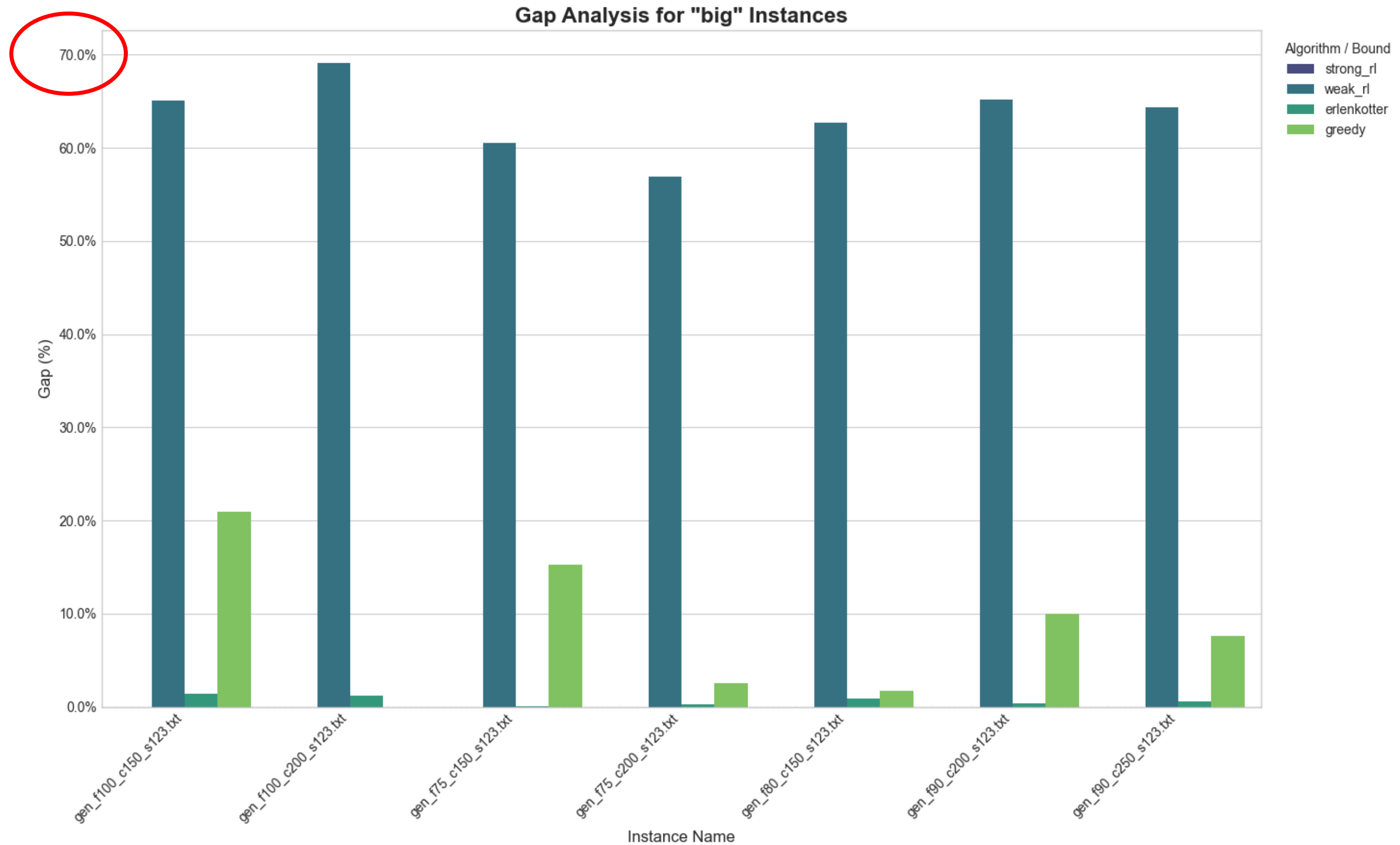


Gerarchia in base alle prestazioni:



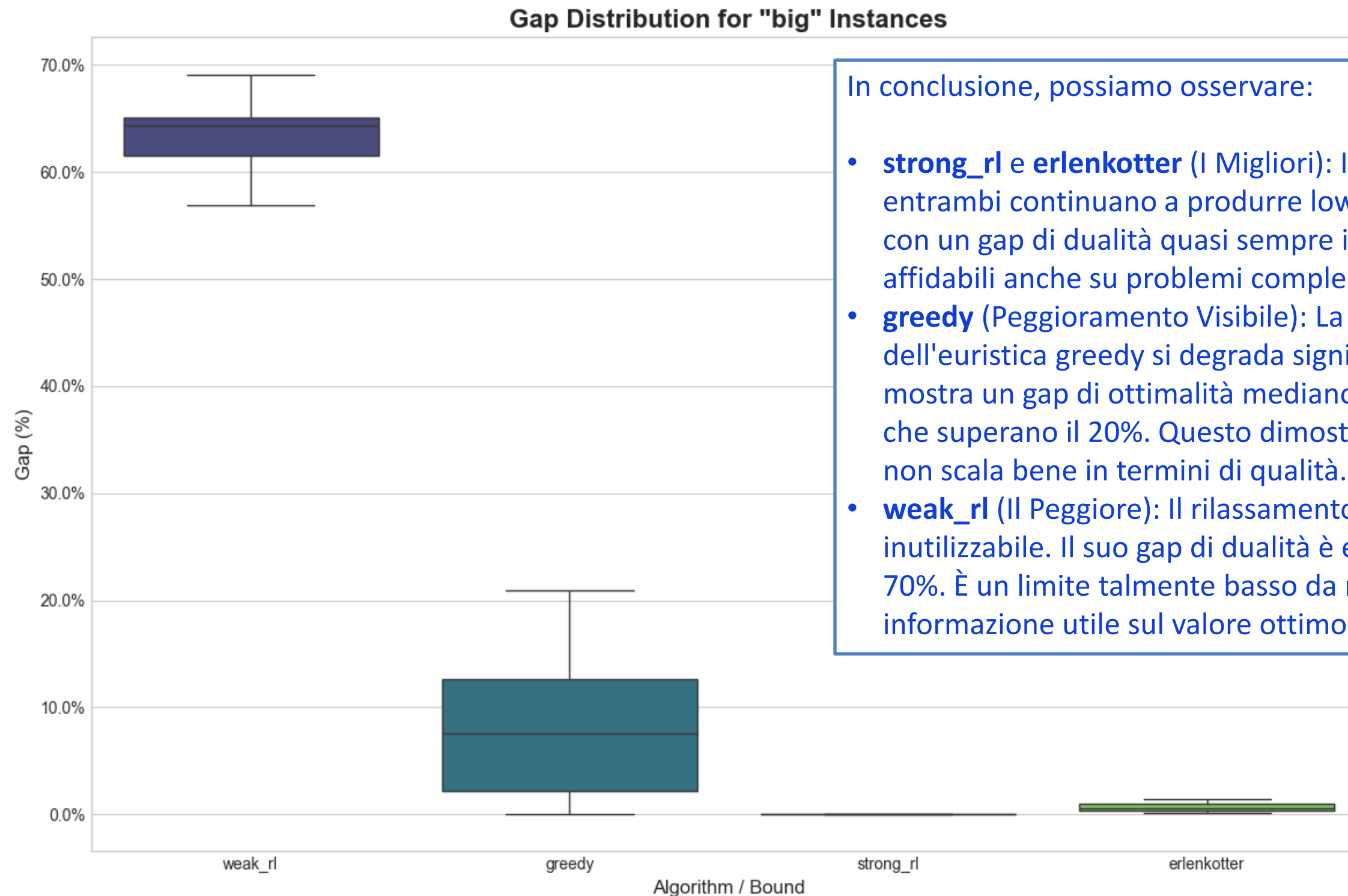


# Analisi di Gap per le istanze big





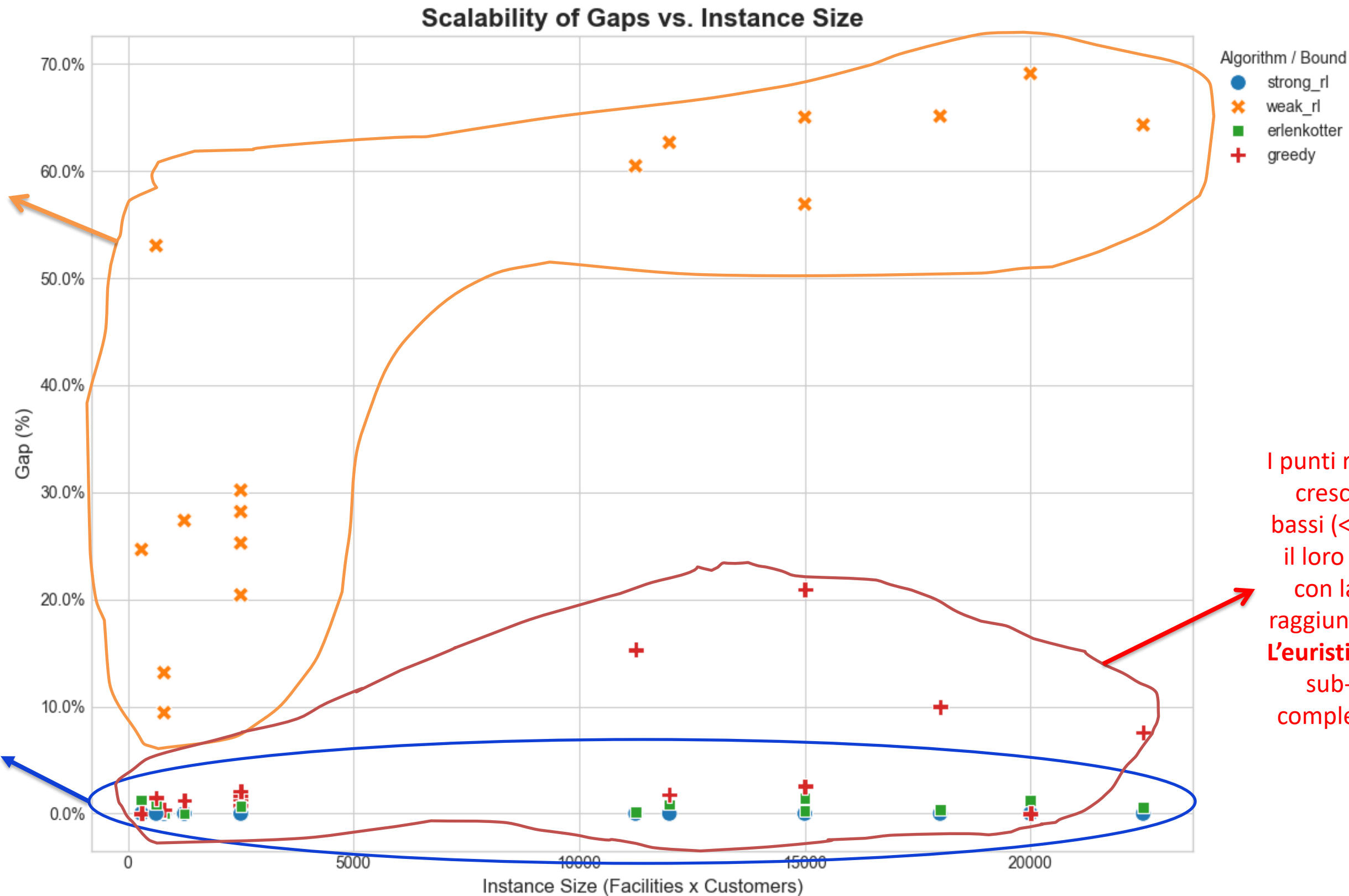
# Distribuzione di Gap per le istanze big



In conclusione, possiamo osservare:

- **strong\_rl** e **erlenkotter** (I Migliori): I grafici mostrano che entrambi continuano a produrre lower bound di altissima qualità, con un gap di dualità quasi sempre inferiore al 2%. Sono stabili e affidabili anche su problemi complessi.
- **greedy** (Peggioramento Visibile): La qualità dell'upper bound dell'euristica greedy si degrada significativamente. Il box plot mostra un gap di ottimalità mediano intorno al 10%, con picchi che superano il 20%. Questo dimostra che il suo approccio miope non scala bene in termini di qualità.
- **weak\_rl** (Il Peggior): Il rilassamento debole è completamente inutilizzabile. Il suo gap di dualità è enorme e stabile intorno al 60-70%. È un limite talmente basso da non fornire alcuna informazione utile sul valore ottimo.

# Scalabilità di Gap rispetto alle istanze



Il rilassamento debole è inaffidabile e non scala. Il suo gap di dualità è sempre molto grande, rendendolo un pessimo strumento per problemi di dimensioni realistiche.

Il rilassamento forte e l'ascesa duale di Erlenkotter sono estremamente robusti. La qualità del loro lower bound non si degrada con l'aumentare della dimensione del problema, rimanendo quasi sempre entro il 2% dall'ottimo.

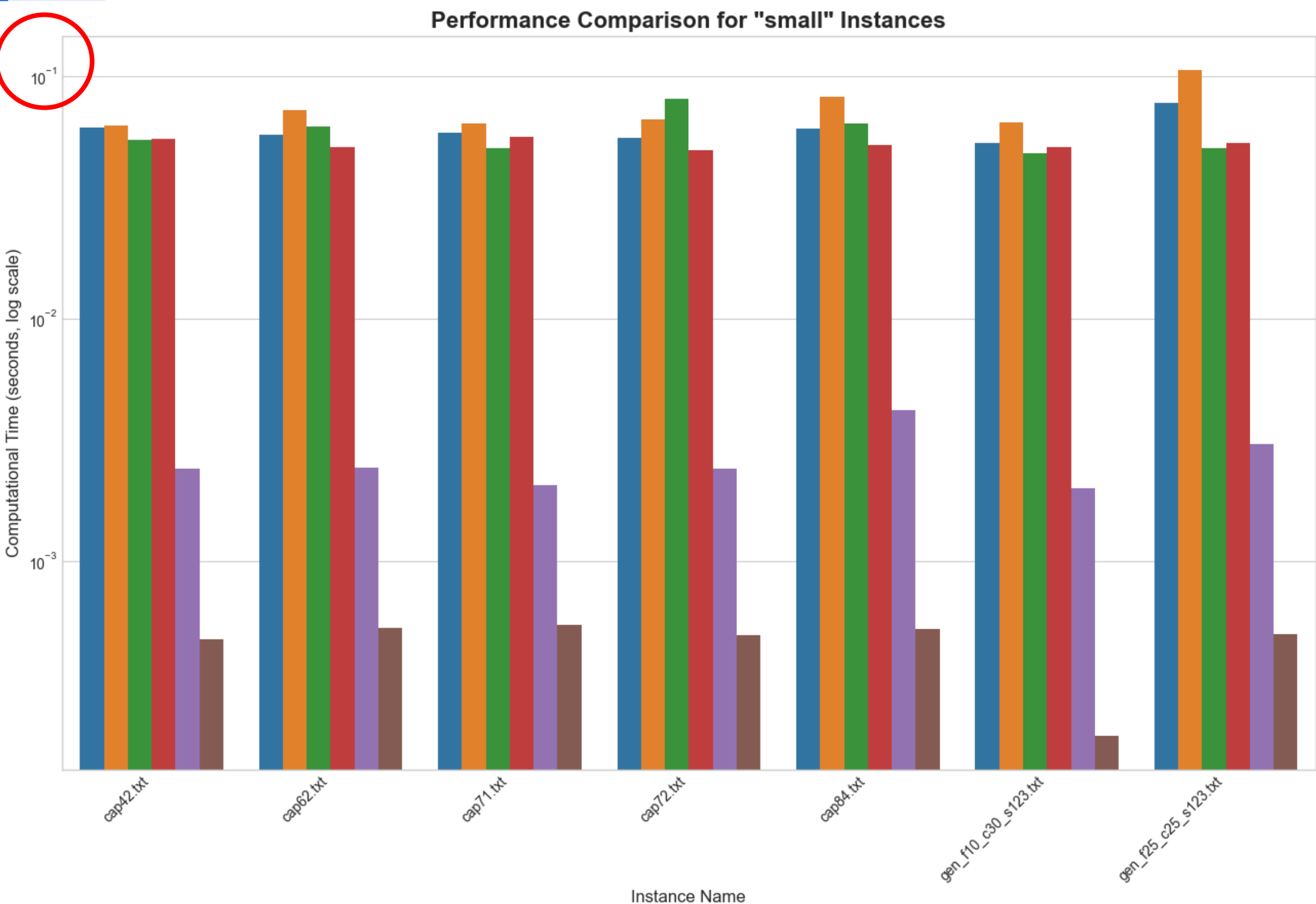
I punti rossi mostrano un chiaro trend crescente. Partono da gap molto bassi (<2%) per le istanze piccole, ma il loro errore aumenta visibilmente con la dimensione del problema, raggiungendo gap del 15-20% e oltre. L'euristica Greedy diventa sempre più sub-ottimale man mano che la complessità del problema aumenta.

# Analisi su tempo di computazione

# Comparazione sul tempo di computazione

## (Istanze small)

Performance Comparison for "small" Instances



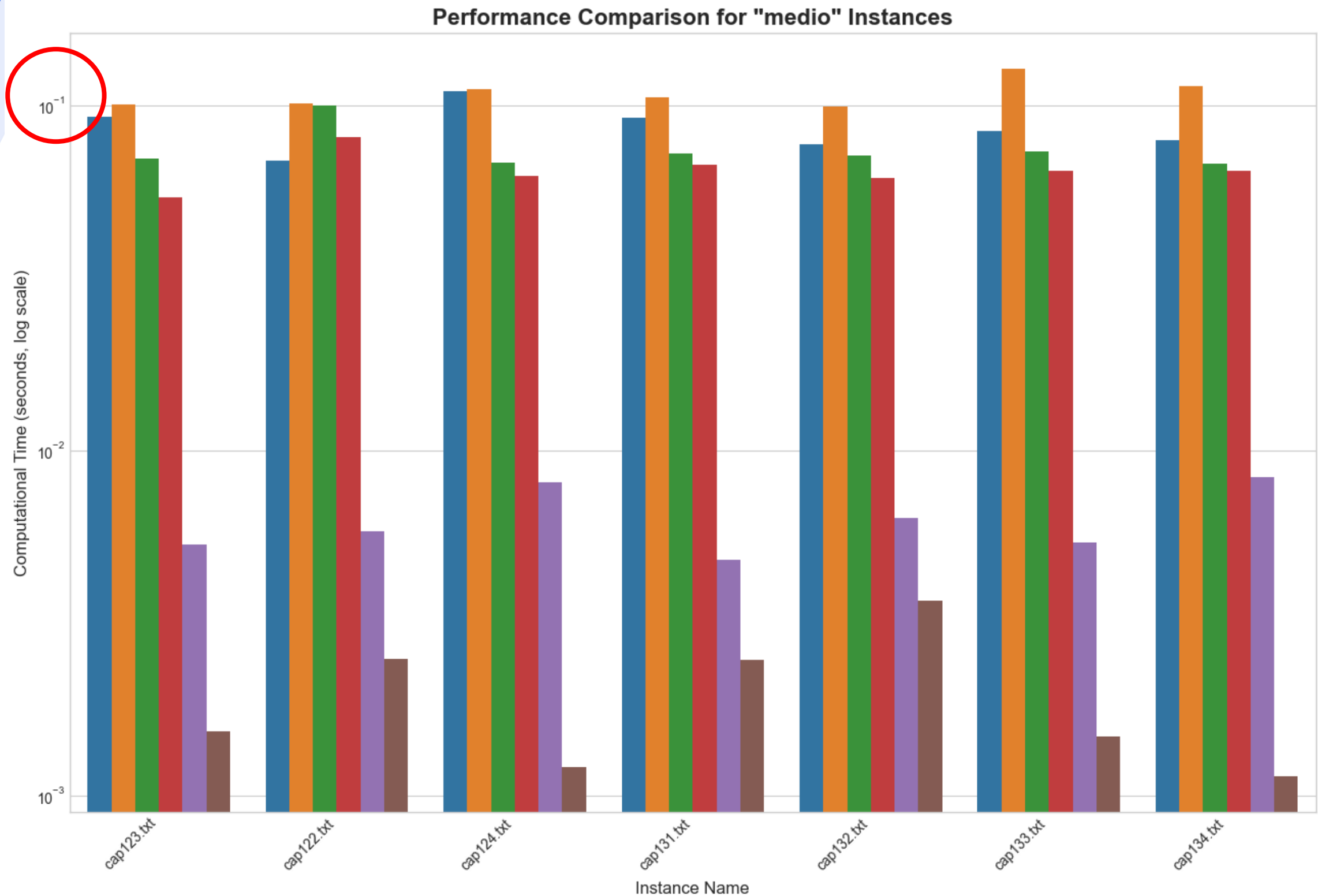
Algorithm

- strong
- weak
- strong\_rl
- weak\_rl
- erlenkotter
- greedy

| Algorithm   | Instances | Mean Time (s) | 95% CI           | Time Range (s)   |
|-------------|-----------|---------------|------------------|------------------|
| strong      | 7         | 0.063905      | [0.0599, 0.0679] | [0.0594, 0.0720] |
| weak        | 7         | 0.096074      | [0.0523, 0.1398] | [0.0640, 0.1999] |
| strong_rl   | 7         | 0.057630      | [0.0545, 0.0607] | [0.0543, 0.0624] |
| weak_rl     | 7         | 0.056899      | [0.0526, 0.0612] | [0.0513, 0.0659] |
| erlenkotter | 7         | 0.002874      | [0.0021, 0.0037] | [0.0019, 0.0043] |
| greedy      | 7         | 0.000480      | [0.0004, 0.0006] | [0.0002, 0.0006] |

# Comparazione sul tempo di computazione

## (Istanze medio)



Algorithm

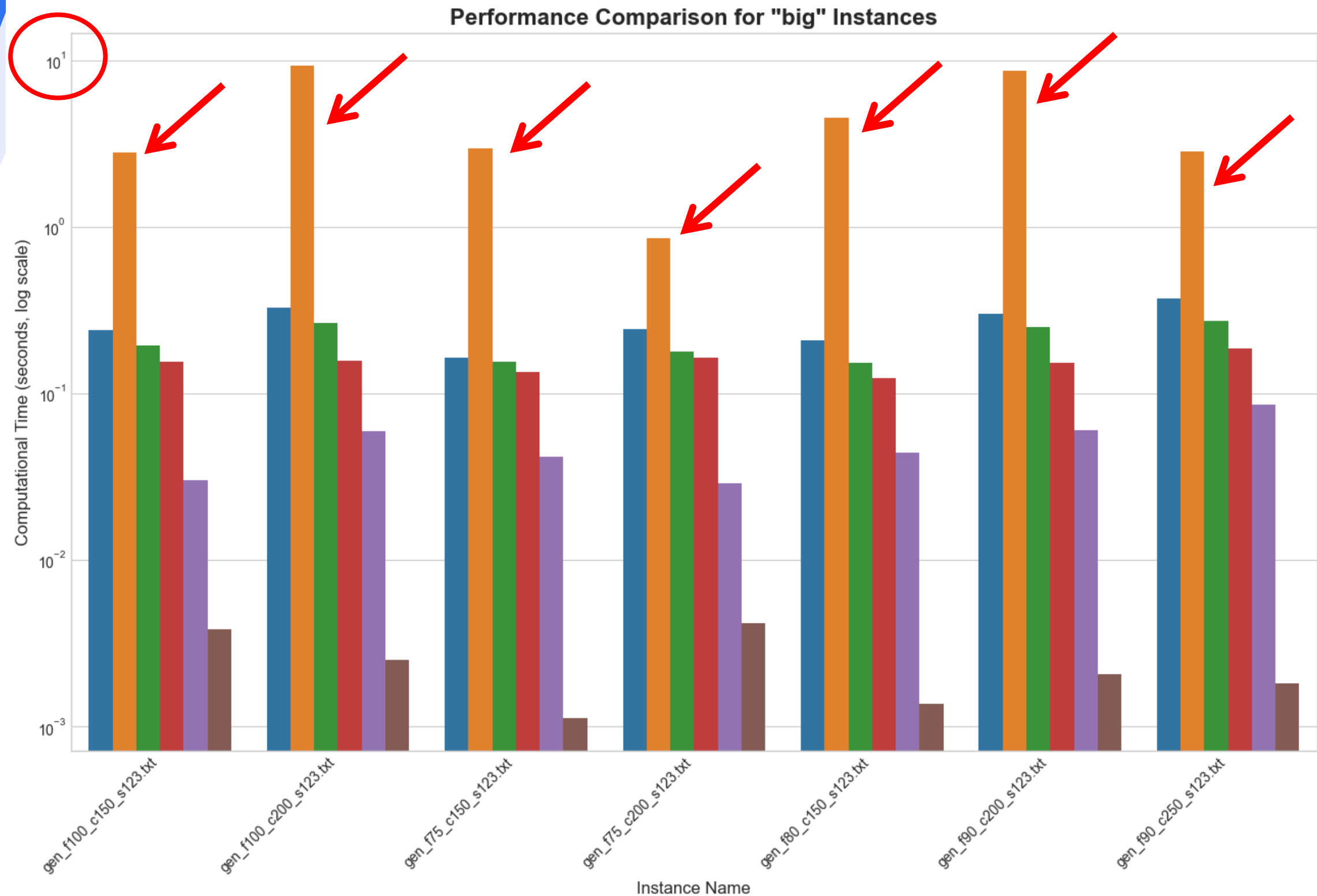
- strong
- weak
- strong\_rl
- weak\_rl
- erlenkotter
- greedy

| Algorithm   | Instances | Mean Time (s) | 95% CI            | Time Range (s)   |
|-------------|-----------|---------------|-------------------|------------------|
| strong      | 7         | 0.080764      | [0.0769, 0.0846]  | [0.0741, 0.0880] |
| weak        | 7         | 0.293072      | [-0.1419, 0.7280] | [0.0994, 1.3591] |
| strong_rl   | 7         | 0.075313      | [0.0674, 0.0832]  | [0.0633, 0.0869] |
| weak_rl     | 7         | 0.066726      | [0.0631, 0.0703]  | [0.0626, 0.0730] |
| erlenkotter | 7         | 0.006272      | [0.0050, 0.0076]  | [0.0045, 0.0082] |
| greedy      | 7         | 0.001727      | [0.0012, 0.0022]  | [0.0010, 0.0023] |



# Comparazione sul tempo di computazione

## (Istanze big)



| Algorithm   | Instances | Mean Time (s) | 95% CI           | Time Range (s)   |
|-------------|-----------|---------------|------------------|------------------|
| strong      | 7         | 0.339261      | [0.1166, 0.5619] | [0.1803, 0.8682] |
| weak        | 7         | 4.363402      | [1.3618, 7.3650] | [0.6931, 8.9413] |
| strong_rl   | 7         | 0.203140      | [0.1586, 0.2476] | [0.1450, 0.2634] |
| weak_rl     | 7         | 0.139506      | [0.1193, 0.1597] | [0.1032, 0.1753] |
| erlenkotter | 7         | 0.053973      | [0.0388, 0.0692] | [0.0391, 0.0851] |
| greedy      | 7         | 0.002557      | [0.0012, 0.0039] | [0.0012, 0.0048] |

Su istanze grandi, la formulazione weak diventa praticamente inutilizzabile.



# CONCLUSIONI

## Modelli Esatti (PLI):

- La **formulazione forte** si è dimostrata nettamente superiore a quella debole. Non solo fornisce un limite inferiore teoricamente più stretto (spesso perfetto), ma questo si traduce in tempi di esecuzione drasticamente inferiori e più stabili su istanze di medie e grandi dimensioni.
- La **formulazione debole**, a causa del suo pessimo rilassamento lineare, è lenta, instabile e inefficace su problemi complessi.

## L'Algoritmo di Erlenkotter è il Miglior Compromesso:

- L'implementazione dell'ascesa duale di Erlenkotter si è rivelata eccezionale. Calcola un lower bound di qualità quasi ottimale ( $\text{gap} < 2\%$ ) con la velocità di un'euristica (pochi millisecondi). Offre il miglior equilibrio tra accuratezza e tempo di calcolo.

## L'euristica greedy ha dei limiti:

- È l'algoritmo più veloce in assoluto, ma la sua qualità della soluzione (upper bound) si degrada all'aumentare della complessità del problema, con gap che diventano significativi.

# Grazie per l'attenzione

LINK GITHUB: [https://github.com/valejin/Progetto\\_AMOD](https://github.com/valejin/Progetto_AMOD)